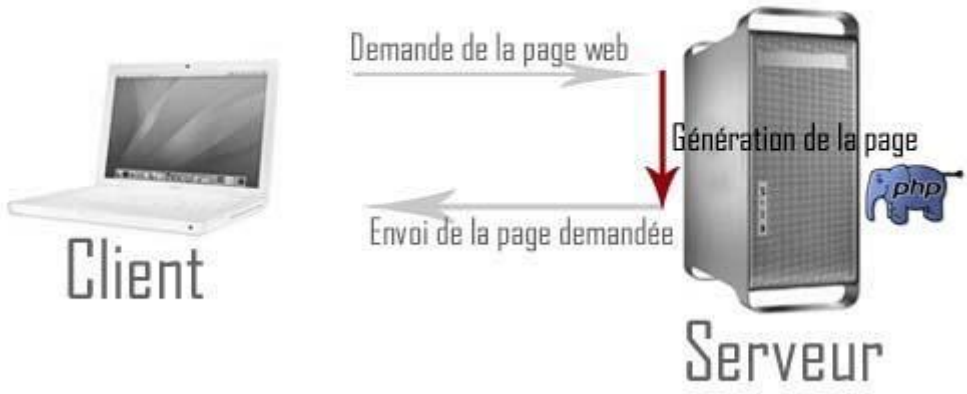
PHP : Qu'est-ce que c'est ?

PHP ou Hypertext Preprocessor est un langage de script. A l'origine, on l'avait conçu pour agir sur les serveurs web.
Avec le XHTML on peut déjà créer des pages web statiques. Le rôle de PHP est de les rendre dynamiques. Grâce à son interaction avec la base de données MySQL et son intégration facile dans du code XHTML, PHP est devenu le langage le plus utilisé en développement web.
PHP permet d'inclure des programmes pour faire de la vente en ligne, un chat, un forum et bien d'autres choses encore dans un site web.
Bien que PHP soit utilisé généralement pour la création de site web, sa capacité n'est pas limitée à ce domaine. Le PHP est open source. **Il est distribué via une licence propre qui précise qu'il peut être utilisé, modifié, utilisé librement et gratuitement** (licence à peu près identique à celle d'Apache).
Vous pouvez télécharger PHP gratuitement depuis le site officiel http://www.php.net. L'autre avantage de PHP est qu'il est facile à prendre en main pour les débutants. De plus, il offre des fonctionnalités avancées pour les experts.

Comment fonctionne le langage PHP

Contrairement au code Javascript , le code PHP est exécuté sur le serveur (on dit alors 'interprété'). Lors de la visite d'une page web, il y a ce que l'on appelle une requête client-serveur. Dans un premier temps le client saisit une adresse à travers un navigateur (Firefox par exemple) et en retour, le navigateur retourne la page recherchée. **Par définition, un serveur web est un "ordinateur" présent sur l'Internet et accessible via le réseau qui héberge la page que vous demandez**. Sur ce serveur on trouve Apache, logiciel apte à traiter les requêtes HTTP que vous envoyez lorsque vous demandez une page web.
Apache va donc chercher le fichier demandé dans son arborescence et renvoie à votre navigateur la page XHTML. Votre navigateur interprète les différents langages se trouvant dans ce fichier (XHTML, JavaScript, CSS, etc. ...) et affiche la page.
En réalité, Il y a une étape qui vient s'ajouter entre la demande (côté client) et l'envoi (côté serveur) d'une requête. Avant l'envoi d'une réponse, la page PHP est générée par le serveur et ensuite seulement, il va la transformer en page XHTML, pour que le client puisse la lire.

Graphique du fonctionnement client serveur de PHP



Installer un outil pour tester PHP en local

L'installation d'un serveur web sur son ordinateur personnel permet de tester ses scripts sans avoir à les renvoyer sans cesse par FTP sur le serveur distant.
L'ordinateur va alors servir à la fois de client et de serveur. Alors que pour tester des pages web html en local il suffit d'ouvrir le fichier dans un navigateur, il faut un serveur web sur votre PC local pour tester une page PHP.
Pour cela si on travaille sur Windows, on utilise un des nombreux utilitaires très pratiques qui installeront Apache (le serveur permettant de distribuer des pages web), PHP,(une sorte de module Apache, permettant de dynamiser les pages), MYSQL (le système de gestion

de base de données), et si on travaille sur linux, il y a fort à parier qu'Apache soit déjà automatiquement installé.

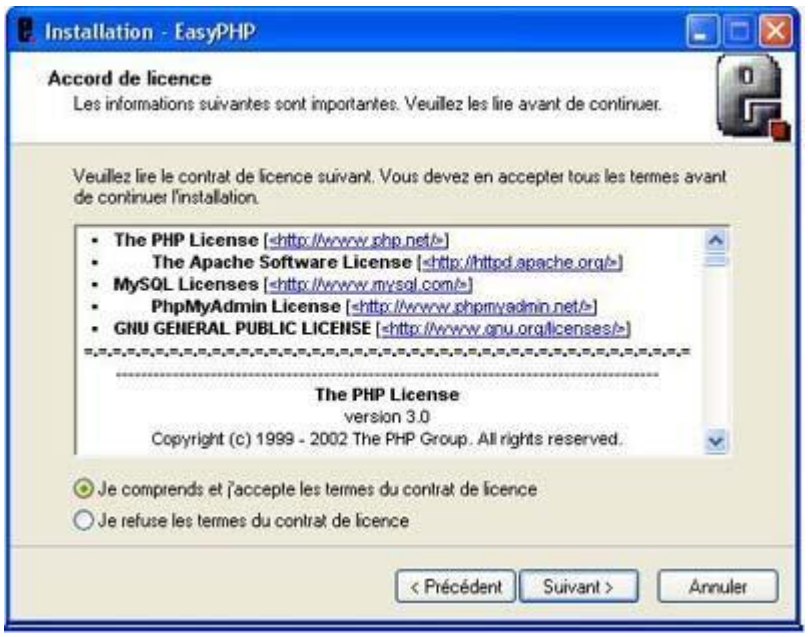
Les outils de développement : EasyPHP et WAMP

Les outils les plus connus sont : EasyPHP, WAMP... Pour le codage, il existe plusieurs sortes d'éditeur mais un simple éditeur de texte peut déjà suffire comme Notepad++
Installer easyPHP en local est très simple comme vous allez le voir.
Voici l'exemple d'easyPHP : il faut lancer l'installation en suivant les instructions suivantes :

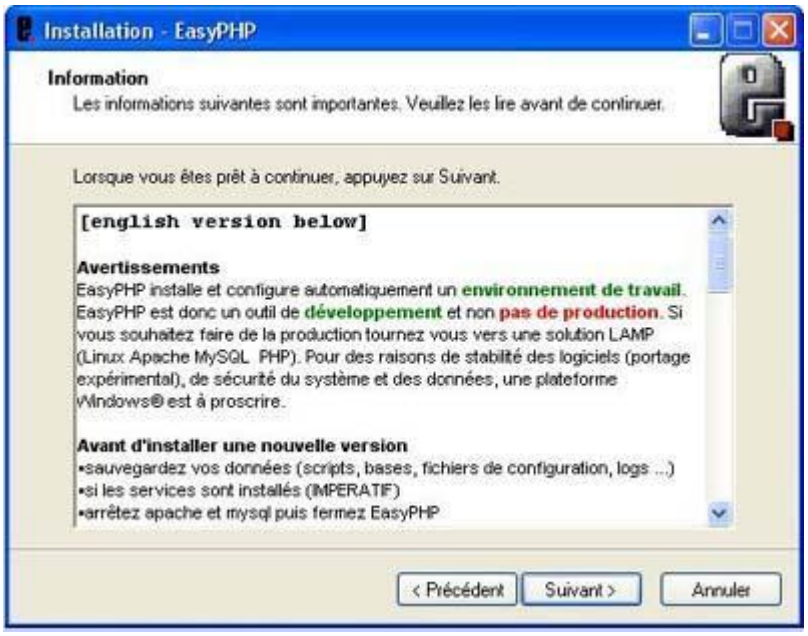
(1)



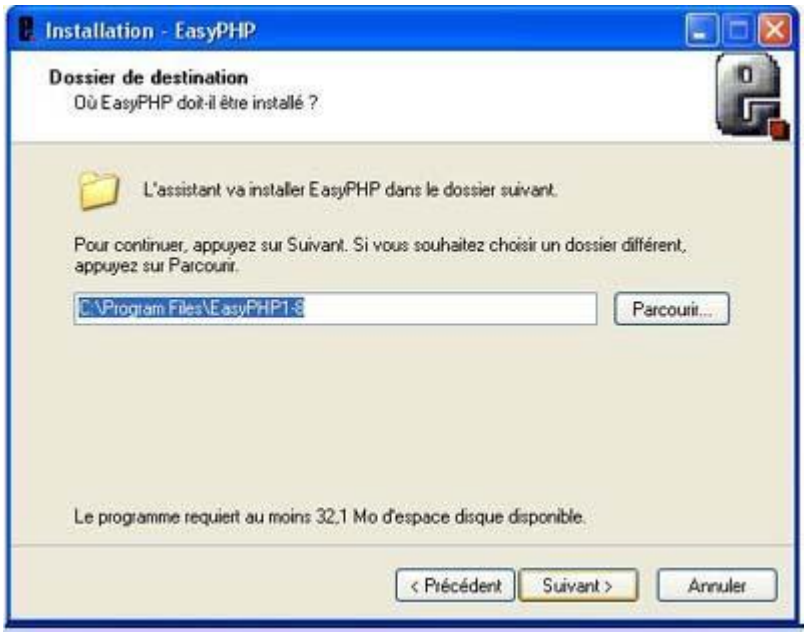
(2)



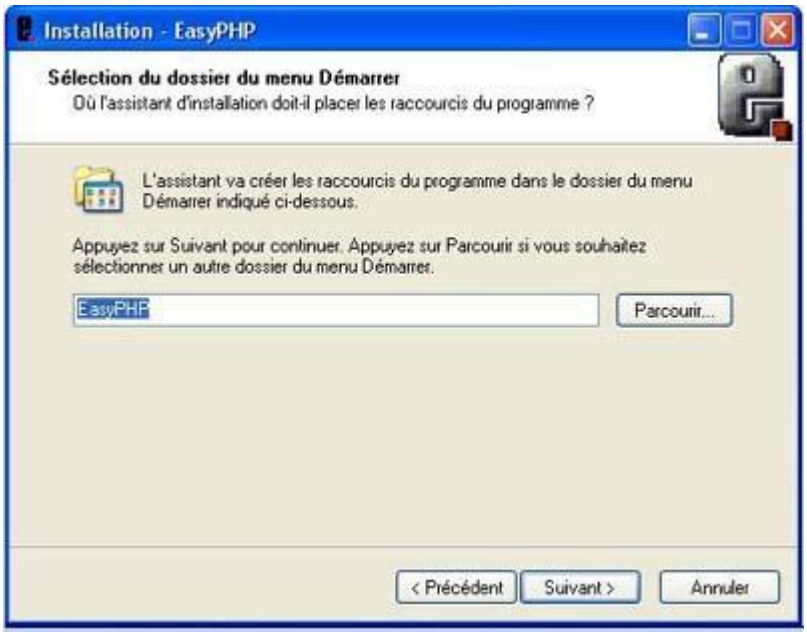
(3)


Installation - EasyPHP
Information
Les informations suivantes sont importantes. Veuillez les lire avant de continuer.
Lorsque vous êtes prêt à continuer, appuyez sur Suivant.
[english version below]
Avertissements
EasyPHP installe et configure automatiquement un environnement de travail.
EasyPHP est donc un outil de développement et non pas de production. Si
vous souhaitez faire de la production tournez vous vers une solution LAMP
(Linux Apache MySQL PHP). Pour des raisons de stabilité des logiciels (portage
expérimental), de sécurité du système et des données, une plateforme
Windows® est à proscrire.
Avant d'installer une nouvelle version
•sauvegardez vos données (scripts, bases, fichiers de configuration, logs ...)
•si les services sont installés (IMPERATIF)
•arrêtez apache et mysql puis fermez EasyPHP
< Précédent
Suivant >
Annuler

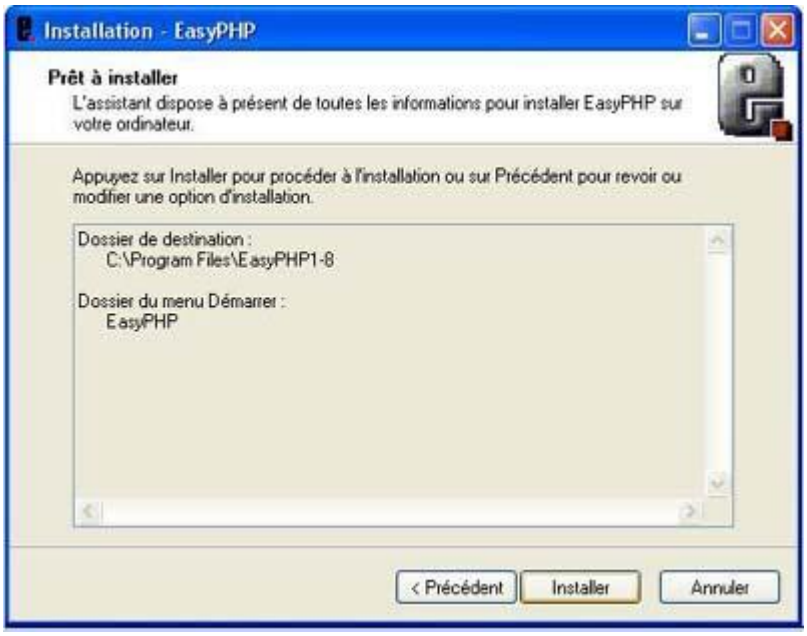
(4)

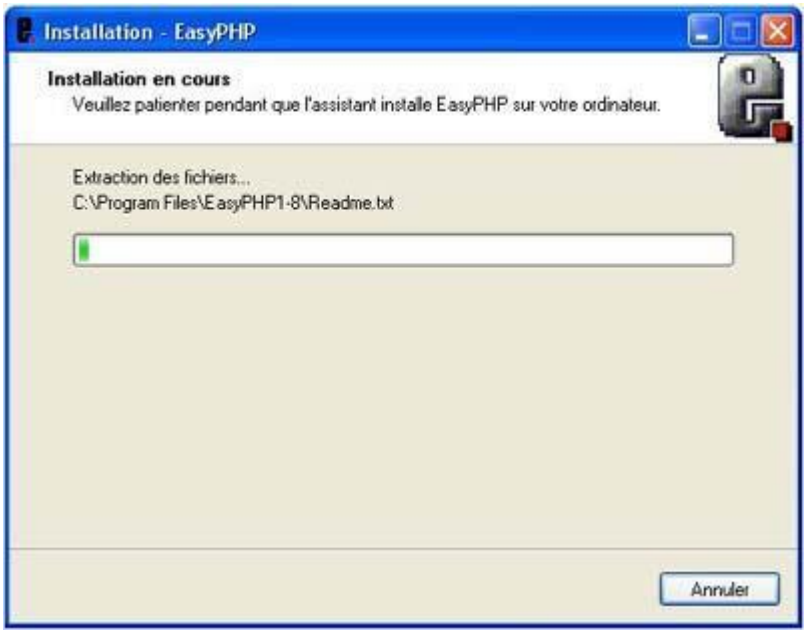

Installation - EasyPHP
Dossier de destination
Où EasyPHP doit-il être installé ?
L'assistant va installer EasyPHP dans le dossier suivant.
Pour continuer, appuyez sur Suivant. Si vous souhaitez choisir un dossier différent,
appuyez sur Parcourir.
C:\Program Files\EasyPHP1-8
Parcourir...
Le programme requiert au moins 32,1 Mo d'espace disque disponible.
< Précédent
Suivant >
Annuler

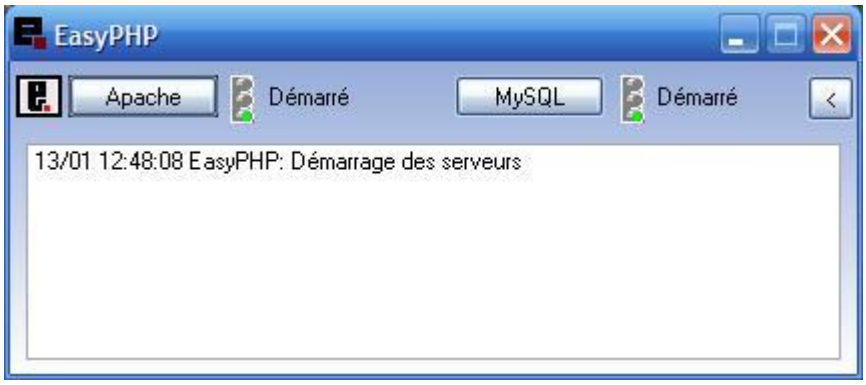
(5)


Installation - EasyPHP
Sélection du dossier du menu Démarrer
Où l'assistant d'installation doit-il placer les raccourcis du programme ?
L'assistant va créer les raccourcis du programme dans le dossier du menu Démarrer indiqué ci-dessous.
Appuyez sur Suivant pour continuer. Appuyez sur Parcourir si vous souhaitez sélectionner un autre dossier du menu Démarrer.
EasyPHP
Parcourir...
< Précédent
Suivant >
Annuler

(6)


Installation - EasyPHP
Prêt à installer
L'assistant dispose à présent de toutes les informations pour installer EasyPHP sur votre ordinateur.
Appuyez sur Installer pour procéder à l'installation ou sur Précédent pour revoir ou modifier une option d'installation.
Dossier de destination :
C:\Program Files\EasyPHP1-8
Dossier du menu Démarrer :
EasyPHP
< Précédent
Installer
Annuler

(7) 

(8) 

Lorsque vous voyez clignoter cette image :



sur la barre d'état alors ça veut dire qu'EasyPHP est démarré et l'installation est alors terminée. A vous de coder...

Premiers pas pour développer avec PHP

Vous venez **d'installer easyPHP** ou autre chose afin de pouvoir développer en PHP ? Vous êtes prêt à débuter avec ce langage de développement ? Alors allons-y !

Structure d'un programme PHP

Pour éditer du code PHP, il existe des éditeurs spécialisés comme Zend Studio, PhpEd, etc. **Au cas où vous n'avez aucun de ces éditeurs, un simple bloc note suffit**.
Si le code HTML est placé entre une balise ouvrante <html> et une balise fermante < /html>, il en est de même pour le PHP, sauf que la balise de début est: <?php et la balise de fin est ?>.
Ceci permet d'indiquer au serveur qu'il s'agit bien d'un code PHP. Afin de mieux comprendre au mieux, voici un premier exemple de ce que pourrait être un programme PHP.

```
<?php
echo "Notre premier test";
?>
```

Pour le tester, il faudra enregistrer la page avec l'extension .php, par exemple "premierprogramme.php", dans un dossier "tests" à créer au préalable (en fonction de votre configuration) dans C:\Program Files\EasyPHP1-8\www . Pour le test de la page, il suffit de démarrer EasyPHP.

Puis allez dans le menu "Localhost", la page d'accueil s'ouvre. Là, si vous avez bien créé votre dossier contenant votre code en PHP dans le répertoire www comme indiqué, vous devriez voir un lien vers votre dossier créé. Cliquez dessus.
Une page web s'ouvre indiquant tous les fichiers qui se trouvent dans le dossier "tests". Vous devriez avoir le fichier " premierprogramme.php", en cliquant sur ce fichier: votre ordinateur génère alors le code PHP puis ouvre la page. Vous avez le résultat de votre code.
Pour faire plus simple, vous pouvez aller directement à cette URL :
http://localhost/tests/premierprogramme.php

Affichage de texte en PHP

L'exemple ci-dessous nous affiche le texte "Notre premier test" à l'écran. Nous devons cela à la fonction echo. Notez bien les guillemets contenant le texte à afficher.
On met toujours le texte entre guillemets, ça permet de repérer ce qu'on veut afficher.
Deux choix nous sont offerts : mettre un guillemet simple ou mettre un guillemet double.
Cependant, il faut faire bien attention à ne pas fermer un guillemet double avec un guillemet simple.
La fonction echo peut prendre plusieurs paramètres séparés par des virgules

```
<?php
echo ' un texte ',' et un autre texte';
?>
```

Il y aussi une autre fonction pour l'affichage. Il s'agit de la fonction print(). Elle donne le même résultat que celui de la fonction echo sauf qu'elle a besoin de parenthèses.

```
<?php
print (' un texte ');
?>
```

Le séparateur d'instructions

Toujours dans le même exemple, nous appelons la partie: echo "Notre premier test"; une instruction. A la fin d'une phrase dans un texte, il y a un point pour terminer la phrase. De même, les instructions en php sont séparées par un point virgule. Ce point virgule indique la fin de chaque instruction.

```
<?php
echo "instruction 1 et";
echo "instruction 2" ;
?>
```

Essayez d'enlever le point virgule à la fin et vous verrez qu'il y aura un message d'erreur "Parse Error".

Les caractères spéciaux

Maintenant, parlons d'une chose très spéciale qui mérite une grande attention.
On avait dit qu'il faut entourer de guillemets les chaînes de caractère à afficher. Mais si le texte à afficher contient lui-même des guillemets. **Comment la fonction echo peut savoir qu'il ne s'agit pas encore de la fermeture de guillemets** ? Evidement, elle sera dans l'embarras si on ne lui indique rien. En effet, une instruction comme celle-ci:

```
<?php
echo ' Voici l'instruction ';
?>
```

Donnera à l'écran une erreur : *Parse error: syntax error, unexpected T_STRING, expecting ',' or ';' in chemin\fichier.php on line 2*
On peut échapper à ce genre de problème, en mettant tout simplement des antislashs. En voici alors une version corrigée de l'instruction précédente :

```
<?php
echo ' Voici l\'instruction ';
?>
```

On fait pareil au cas où il y aurait des "" ou des \ à l'intérieur des textes à afficher.

Ajouter des commentaires dans un script PHP

Il n'est pas obligatoire mais vivement conseillé de mettre des commentaires dans vos codes. Vous pouvez utiliser les commentaires pour expliquer votre code.
Supposons qu'il y très longtemps que vous n'ayez pas touché à vos codes. En revenant de vacances, vous décidez de les reprendre. Avec une centaine de lignes de code, **il est tout à fait probable que vous galériez un peu si vous ne vous souvenez pas à quoi sert telle ou telle fonction.**.

Afin d'éviter la perte de temps dans ce genre de situations, ça ne vous coûte rien de mettre des commentaires dans vos codes. De plus, **si vous travaillez à plusieurs sur un même projet, les autres comprendront plus rapidement votre code** en lisant les commentaires.
Ceci étant dit, comment faire du commentaire alors ? Très simple, il y a deux façons :
Pour un commentaire sur une ligne, il suffit d'ajouter deux slash //

```
<?php
echo ' Voici l\'instruction '; //affichage du texte :Voici l'instruction
?>
```

Et pour ajouter des commentaires sur plusieurs lignes, mettez-les entre /* et */

```
<?php
/* Ceci un programme php pour l'affichage d'un texte.
Et aussi sur les caractères spéciaux.
Désormais je commente mon code.
echo ' Voici l\'instruction '; //affichage du texte : Voici l'instruction
?>
```

Mais attention, il ne faut surtout pas imbriquer les commentaires comme ceci :

```
/* commentaire /*
```

Expressions, variables et types en PHP

Ce tutoriel présente les notions d'expressions, de variables et de types dans le langage PHP. Ces notions que l'on retrouve dans de nombreux langages de développement sont assez simples à assimiler et indispensables pour poursuivre votre apprentissage.

Qu'est-ce qu'une variable ?

Une variable est un objet ou une information stockée en mémoire temporairement. C'est-à-dire qu'en PHP, la variable existe tant que la page est en cours de génération. **Après l'exécution du programme, toutes les variables sont supprimées de la mémoire car elles ne servent plus à rien**.
Une variable est caractérisée par son nom. Donc il est préférable de lui donner un nom significatif pour savoir de quoi il s'agit. En revanche, il faut noter que le nom de variable est sensible à la casse (majuscule / minuscule).
Un nom de variable valide doit commencer par une lettre ou un souligné (_), suivi de lettres, chiffres ou soulignés. La déclaration d'une variable se fait par l'écriture du symbole '$' devant la variable à manipuler.
Exemple :

```
$prix
```

Le nom de cette variable est prix.

Affectation d'une valeur à une variable en PHP

A une variable, on assigne une valeur. L'instruction d'affectation permet d'affecter une valeur à une variable. Il suffit de mettre un signe égal après la déclaration de la variable et de mettre la valeur associée comme suit.

```
$prix = 10; //La variable prix a comme valeur 10
```

Comme toute instruction PHP, il ne faut jamais oublier le signe; (point virgule) à la fin.

Les Types de variables en PHP

Selon la valeur qu'elle va contenir, une variable a un type. Contrairement à de nombreux autres langages, **il n'est pas obligatoire en PHP de préciser les types**. Toutefois, il existe plusieurs types de variables en PHP dont les booléens, les réels, les entiers, les chaînes de caractères, les tableaux.

Les Booléens

Une variable de type booléen ou boolean a deux valeurs constantes possibles : TRUE ou FALSE. Ces constantes sont insensibles à la casse. **Il s'agit donc d'une variable qui ne peut être que true ou false et rien d'autre**.
Exemple :

```
<?php
$jeune = true;
?>
```

Les Entiers

Un entier ou integer est un nombre appartenant à l'ensemble des entiers Z: Z = {…,-1,0, 1, 2, ...}.

Exemple :

```
<?php
$age = 10;
?>
```

Les Réels

Ce sont les nombres décimaux connus aussi sous le vocable de " double ", " float " ou " nombre réels. Un prix peut par exemple être un réel.
Exemple:

```
<?php
$prix = 2.542;
?>
```

Les chaînes de caractères

Ce sont les variables de type texte, ou string. Comme l'on avait dit précédemment, il faut les mettre entre guillemet. Les chaînes de caractères sont des séquences de caractères.
Exemple :

```
<?php
$nom = "Dupont";
?>
```

Il y a encore d'autres types de variables comme les tableaux, les objets, etc, mais qui ne seront pas abordés dans cette section.

Les Constantes

Il s'agit des variables dont la valeur ne pourra être changée lors de l'exécution d'un programme. On utilise donc le type constante pour les valeurs qui ne seront pas susceptibles de changer au cours de l'exécution du programme. **C'est le cas du nombre pi par exemple, ou de la TVA**, etc
La fonction utilisée pour la définition d'une constante en PHP est la fonction define() qui nécessite en paramètres le nom de la variable et sa valeur. La syntaxe de la fonction define() est la suivante: define("Nom_de_la_constante", Valeur);
Exemple :

```
<?php
define("TVA"', 19.6);
echo TVA;
?>
```

Tout au long de l'exécution la valeur de TVA utilisée sera 19.8. Quand on utilise la constante TVA, on ne met plus de guillemet.
La différence entre les constantes et les variables PHP est qu'elles ne sont pas précédées du signe $. Par convention, on écrit les constantes en majuscules.

Les conditions en PHP

Une structure conditionnelle permet d'exécuter ou non une série d'instructions en fonction d'une condition d'origine. Si le calcul de cette condition retourne TRUE alors le bloc d'instructions concerné est exécuté.
Les expressions évaluées peuvent être plus ou moins complexes, c'est-à-dire qu'elles peuvent être constituées d'une combinaison d'opérateurs de comparaison, d'opérateurs logiques et même de fonctions. Le langage PHP introduit 4 constructions conditionnelles: if, elseif, else et switch.
Avant de les voir, il est nécessaire que l'on aborde les opérateurs de comparaison et les opérateurs logiques.

Les opérateurs de comparaison

Les opérateurs de comparaison sont utilisés pour comparer deux valeurs. Elles permettent souvent de définir des conditions dans les structures conditionnelles.
Egal : $a == $b retourne true si la valeur de $a est égale à celle de $b
Différent : $a != $b retourne true si la valeur de $a est différente de celle de $b
$a <> $b retourne true aussi si la valeur de $a est différente de celle de $b
Inférieure : $a < $b retourne true aussi si la valeur de $a est strictement inférieure à celle de $b
Inférieure ou égal : $a < = $b retourne true aussi si la valeur de $a est inférieure ou égale à celle de $b
Supérieure : $a > $b retourne true aussi si la valeur de $a est strictement supérieure à celle de $b

Supérieure ou égal : $a >= $b retourne true aussi si la valeur de $a est supérieure ou égale à celle de $b Les opérateurs logiques
En PHP, il y a différents opérateurs logiques qui permettent de combiner des conditions entre elles.

La structure conditionnelle if, else

Cette structure permet de n'exécuter qu'un bloc d'instructions uniquement si l'expression est vraie. Le mot clé if() signifie en anglais 'si'. Autrement dit, si la condition est vérifiée, on exécute l'instruction. Dans le cas contraire, l'instruction sera simplement ignorée ou une autre instruction alternative sera exécutée.
Par conséquent, un if peut être employé seul. Le else étant l'alternative, on peut s'en passer pour n'exécuter un code que si une condition est réalisée.
La syntaxe est :

```
<?php
if(condition)
{
  // Bloc d'instructions
}
?>
```

On met la ou les conditions entre parenthèses, et le bloc d'instructions entre deux accolades.
Exemple :

```
<?php
$vitesse = 100;

if($vitesse > 50)
{
  echo "excès de vitesse !";
}
?>
```

Else

La clause else (traduite par sinon), ajoutée après l'accolade fermante du bloc if(), permet de définir une série d'instructions qui seront exécutées si l'expression if testée est fausse (c'est-à-dire si elle renvoie FALSE).
Voici la syntaxe :

```
<?php
if (condition) {
  // instruction au cas où la condition serait réalisée;
}
else{
  // instruction au cas où la condition ne serait pas réalisée;
}
```

Exemple :

```
<?php
$age = 22;

if($age >= 18)
{
  echo "Vous êtes majeur.";
}
else
{
  echo "Vous êtes mineur.";
}
?>
```

Imbrication elseif

On peut imbriquer les if les uns dans les autres. Simplement lorsqu'un if imbriqué aura fini d'être exécuté, il retournera à l'étape logique suivante du rang hiérarchique supérieur.
Exemple :

```
<?php
$nombre = -4;

if($nombre == 0)
```

```
{
  echo "le nombre est égal à zéro";
}
else
{
  if($nombre > 0) {
     echo "le nombre est positif";
  } else {
     echo "le nombre est négatif";
  }
}
?>
```

On peut simplifier le code en utilisant elseif. Et le nombre d'elseif est illimité. Sauf que le else à la fin est obligatoire et il est exécuté lorsqu'aucune des conditions en dessus ne sont pas exécutées. Exemple :

```
<?php
$nombre = -4;
if($nombre == 0)
{
echo "le nombre est égal à zéro";
}
elseif($nombre > 0)
{
  echo "le nombre est positif";
}
else
{
  echo "le nombre est négatif";
}
?>
```

Les boucles en PHP

Les boucles sont des structures de contrôle permettant d'effectuer un certain nombre de fois les mêmes opérations. Concrètement, il s'agit d'une répétition d'instructions.
Par exemple afficher une suite de nombres que l'on incrémentera à chaque tour de boucle en fonction d'une ou plusieurs conditions à vérifier.

Les opérateurs d'incrémentation et de décrémentation

Incrémenter une valeur signifie qu'on augmente la valeur d'un certain ordre. Pour dire que la valeur d'une variable $i a augmenté d'un point, on écrit :

```
$i = $i+1 ;
```

Cette instruction peut être écrite d'une manière plus simple en utilisant une instruction équivalente :

```
$i += 1 ;
```

ou tout simplement :

```
$i ++ ;
```

A l'inverse, décrémenter une valeur veut dire qu'on diminue la valeur de $i d'un certain nombre de points. La syntaxe est la même que celle de l'incrémentation, sauf qu'à la place du signe + (plus) on met un signe – (moins).
Il est tout à fait possible de faire : $i += 2; pour dire que la valeur de $i augmentera de deux points.

Boucle while en PHP

C'est le moyen le plus simple pour faire une boucle. On traduit la boucle while par 'tant que'. Tant que la/les conditions est/sont vérifiée/s, on traite les instructions situées dans la boucle. La particularité de cette instruction est que la condition est testée à chaque début de boucle.
Syntaxe de while :

```
<?php
while(condition)
{
  // instruction 1;
  // instruction 2;
```

```
  ...
} ?>
```

Dans cet exemple, on affichera tous les nombres pairs qui sont inférieurs à 20.
Exemple :

```
<?php
$i = 0; // $i est un nombre que l'on incrémentera.
while($i < 20)
{
  echo $i . "<br />";
  $i += 2 ;
}
?>
```

L'essentiel à retenir est que la boucle while() signifie que l'on va répéter un bloc d'instructions tant que la condition passée en paramètre reste vraie (TRUE). Lorsque celle-ci deviendra fausse (FALSE), le programme sortira de la boucle.
Il est donc impératif que le bloc d'instructions, d'une manière ou d'une autre, agisse sur le paramètre. Dans l'exemple précédent, si l'on ne met pas la ligne $i += 2, alors la boucle ne s'arrêtera pas !

Boucle for en PHP

La boucle for() se traduit par 'pour chaque itération'. Il faut connaître par avance la condition d'arrêt. Autrement dit, la valeur qui rendra la condition fausse et stoppera la boucle. Sa syntaxe est simple et prend 3 paramètres obligatoires:
Syntaxe de for

```
<?php
for(initialisation; condition; incrémentation)
{
  bloc d'instructions;
}
?>
```

L'initialisation est l'expression qui permet d'initialiser la boucle (valeur de départ). Généralement, les boucles débutent à la valeur 0, mais ce n'est pas une généralité. Le second paramètre correspond à la condition d'arrêt de la boucle.
Cette condition est recalculée à chaque itération (passage de boucle) afin de déterminer si l'on continue de boucler ou bien si l'on sort de la boucle. Enfin, le dernier paramètre détermine l'expression qui sera exécutée à la fin d'une itération. Généralement on prévoit ici une incrémentation (ou décrémentation) pour mettre à jour le compteur de la boucle.
Exemple :

```
<?php
// Boucle générant la table de multiplication du 8
for($i=0; $i<=10; $i++)
{
  echo "8 x " . $i . " = " . (8*$i) . "<br/>";
}
?>
```

La boucle do-while en PHP

L'instruction do{ ... } while() traduite par: 'répéter / faire ... tant que' est une alternative à l'instruction while().
Elle permet de tester la condition après la première itération et exécution du premier bloc d'instructions. **Dans le cas de la boucle while, la condition est examinée avant la boucle** tandis que pour la boucle do-while elle est examinée à la fin. Ainsi, même si cette condition n'est pas vérifiée, la boucle s'exécutera au moins une fois.
Syntaxe de do while :

```
<?php
do
{
  // bloc d'instructions;
}
while(condition); ?>
```

Exemple :

```php
<?php
// Déclaration et initialisation du compteur
$i = 0;
// Boucle générant la table de multiplication du 8
do
{
  echo "8 x ". $i . " = " . (8*$i) . "<br/>";
  // Incrémentation du compteur
  $i++;
}
while($i <= 10);
?>
```

Remarques: Il existe deux instructions qui permettent de modifier l'exécution des boucles, il s'agit de break et de continue.
L'instruction break permet de sortir de la boucle courante. C'est-à-dire qu'il interrompe directement l'itération. La condition de boucle est une condition qui est toujours vérifiée et, dans la boucle, on utilise break pour quitter celle-ci une fois que l'on est arrivé à nos fins.
'break' permet aussi de gérer des événements plus exceptionnels, comme des erreurs: en cas d'erreur, on quitte la boucle et on affiche un message.
La seconde instruction est l'instruction continue. **Cette instruction permet de sauter les instructions de l'itération courante, afin de passer directement à l'itération suivante**.
Reprenons l'exemple sur les nombres pairs. Cette fois-ci, la variable $i sera incrémentée de 1. On utilisera l'instruction continue pour éviter les nombres impairs.

```php
<?php
$i = 0; // $i est un nombre que l'on incrémentera.
while($i < 20)
{
 if (! ($i % 2) )
 {
  continue;
 }
echo $i . "<br />";
$i+=1 ;
}
?>
```

Syntaxe de l'instruction conditionnelle Switch

Il existe une autre alternative à la structure if() / elseif() / else ou bien aux imbrications de blocs if(). Elle se nomme switch() (traduit par 'au cas où').
Sa syntaxe est assez simple et repose sur l'utilisation de 3 mots clés : switch, case et default.
Cette instruction conditionnelle permet de tester toutes les valeurs possibles que peut prendre une variable.

Exemple d'utilisation de l'instruction switch en PHP

```php
<?php

$legume = "rien";

switch($legume)
{
 case 'salade':
  echo'Vous avez acheté de la salade !';
  break;

 case 'Carotte':
  echo'Vous avez acheté de la Carotte !';
  break;

 case 'poivrons':
  echo'Vous avez acheté des poivrons!';
  break;

 case 'aubergines':
  echo'Vous avez acheté des aubergines!';
  break;
```

```
 default :
  echo 'Vous avez acheté un autre légume' ;
  break;
}

?>
```

Dans cet exemple, $legume est la variable à tester.
Les différents 'case' testent la valeur, et exécutent le code contenu entre le 'case' en question et le 'break'.
L'instruction contenue dans la clause default est l'instruction à exécuter par défaut lorsque la variable $legume ne prend aucune des valeurs définies dans les différents 'case'.

Les fonctions en PHP

Une fonction est un bloc de code PHP destiné généralement à être réutilisé plusieurs fois. **Plutôt que d'écrire plusieurs fois le même morceau de code, on met celui-ci dans une fonction**, et c'est cette fonction que l'on appellera dès que l'on en aura besoin.
En PHP, il y a déjà des fonctions prédéfinies que l'on peut utiliser immédiatement. Pour autant, rien ne nous empêche d'en créer d'autres selon nos besoins. Et plus encore, on peut se servir des fonctions PHP pour créer nos propres fonctions.

Exemple de fonctions PHP prédéfinies

```
<?php
// Pour connaître la version de PHP utilisé on écrit :
echo phpversion();
?>
```

Création d'une fonction en PHP

Définition de la fonction
Lors de la définition d'une fonction, on utilise en premier le mot clé 'function'. Il est obligatoire de le mettre pour dire que le bloc traité par PHP est une fonction. Ensuite, on met le nom de la fonction.
Une fonction peut ne pas retourner une valeur. Dans ce cas, on parle généralement de procédure. C'est juste pour structurer le code et pour alléger le code en évitant les répétitions.
Exemple de procédure affichant "bonjour le monde !":

```
<?php
function hello()
 {
  echo "Bonjour tout le monde !";
 }

hello(); //appelle la fonction "hello" qui va afficher "Bonjour tout le monde !" (sans les guillemets)
?>
```

Pour afficher Bonjour le monde ! n'importe où dans le code, il n'y aura plus qu'à taper cec i:

```
hello();
```

Mais la plus grande utilité des fonctions vient du fait qu'elles peuvent retourner une valeur. On peut se servir d'une fonction pour manipuler des variables. Au départ, on leur introduit des valeurs qu'on appelle les arguments de la fonction.
Le bloc de fonction est donc devenu comme une sorte de machine dans laquelle on manipule les paramètres. Après manipulation, la machine (en l'occurrence la fonction) retourne une valeur. Toutes les manipulations se déroulent à l'intérieur du bloc de fonction.
Nous venons de voir l'exemple d'une procédure qui ne contenait aucun paramètre. Une fonction peut avoir un ou plusieurs paramètres. Ces paramètres sont généralement des variables et peuvent être de types différents. Après avoir écrit le nom de la fonction, on énonce tous les paramètres entre les deux parenthèses.
En cas de plusieurs paramètres il faut les séparer par des virgules.
Voici un exemple de fonction simple qui calcule la division de 2 nombres. On met les deux nombres en paramètres. Le nombre retourné par la fonction est donc le résultat de la division des deux nombres. Le mot clé 'return' permet de retourner le résultat.

```
<?php
function division($nombre1, $nombre2)
{
  $resultat=$nombre1/$nombre2;
```

```
  return $resultat;
}
?>
```

Comment appeler une fonction PHP avec paramètres ?

Pour appeler une fonction, il suffit de l'appeler par son nom et de spécifier les valeurs des paramètres.
Si nous voulons par exemple appeler la fonction division :

```
<?php
function division($nombre1, $nombre2)
{
  $resultat=$nombre1/$nombre2;
  return $resultat ;
}

$valeur=division(100,50);
echo $valeur;
//ou encore
$numerateur=100;
$denominateur=50;
$quotien= division($numerateur, $denominateur);
echo $quotien;
?>
```

Remarquez bien qu'il y a un ordre dans la disposition des paramètres dans la définition de la fonction. **Si l'on intervertit les paramètres, il y a de fortes chances que l'on n'ait pas le résultat attendu.**
Autrement dit, division(100, 50); ne donnera évidemment pas le même résultat que division(50,100);

Valeurs par défaut des paramètres d'une fonction

Nous pouvons également assigner des valeurs par défaut aux paramètres.
Exemple

```
<?php
function division ($nombre1 = 10, $nombre2 = 2)
{
  $resultat=$nombre1/$nombre2;
  return $resultat ;
}
echo division();
?>
```

Ici, nous assignons comme valeurs par défaut les nombres 10 et 2. Si nous ne spécifions pas les valeurs des paramètres lors de l'appel de la fonction, nous obtiendrons 5 comme résultat.
Lorsque l'on met des valeurs par défaut, les deux paramètres ne sont pas obligatoires lors de l'appel de la fonction. Dans nos exemples, on a toujours fait l'appel de la fonction juste en bas de la définition de la fonction. Tout se passe donc dans le même fichier.
Mais lorsqu'on a plusieurs fonctions, et qu'on veut utiliser les mêmes fonctions dans différents fichiers, il devient fastidieux de répéter les définitions des fonctions en haut de chaque fichier. PHPnous offre la possibilité de créer une sorte de bibliothèque de fonctions et de les inclure tout simplement au début de chaque fichier.
C'est ce que nous allons voir dans le chapitre suivant.

Manipuler les chaînes de caractère

Avant d'aborder le vif du sujet, faisons d'abords un petit rappel de ce que l'on a déjà su à propos des chaînes de caractères.

Affichage de chaînes de caractère

Comme nous venons de voir dans le chapitre concernant 'la syntaxe de base en PHP', on utilise deux fonctions pour afficher des chaînes: echo et print. Ensuite, nous avons dit qu'il faut mettre ces chaines entre guillemets doubles ou guillemets simples.

Concaténation de chaînes

L'opération de concaténation des chaînes de caractère est le point. Il suffit donc de mettre un point entre deux chaînes pour les concaténer.

Les chaînes de caractère en tant que variable

Comme toute variable, l'affectation d'une variable chaîne se fait par le symbole "=" . Dès qu'on met un ou plusieurs caractères entre guillemet, PHP pense qu'il s'agit d'une chaîne de caractère.

Pour le tester, nous allons utiliser la fonction 'gettype' qui retourne le type de la variable passée en paramètre. Pour info, la fonction settype() permet, à l'inverse, d'affecter un type précis à une variable.

```
<?php
$nb = 5;
echo gettype($nb);
?>
```

La valeur affichée à l'écran est integer. C'est à dire que la variable $nb est de type nombre.
Dans ce deuxième exemple, même s'il s'agit toujours du nombre 5, la valeur affichée à l'écran n'est plus integer. Il est du type string ou chaine dès qu'on met le guillemet.

```
<?php
$nb = "5";
echo gettype($nb);
?>
```

Longueur d'une chaîne et élagage : Fonction strlen et trim

L'élagage est une opération qui consiste à retirer les blancs avant et après une chaîne de caractère.
La fonction strlen nous permet d'avoir la longueur du texte.
En ce qui concerne l'enlèvement des blancs du début et de la fin de la chaine, il suffit d'utiliser la fonction trim. Notre chaîne de départ est de longueur 33 par exemple:

```
<?php
$chaine = " on étudie le php et etc.... ";
echo strlen($chaine); //renvoie 33
?>
```

En appliquant un trim pour enlever les espaces, il ne nous reste plus que 27 caractères.

```
<?php
$chaine = " on étudie le php et etc.... ";
$chaine = trim($chaine);
echo $chaine."<br />";
echo strlen($chaine);//renvoie 27
?>
```

On peut aussi enlever d'autres caractères que l'espace (comme les points par exemple) en donnant un deuxième paramètre à la fonction trim. On repassant une deuxième fois la fonction trim avec un point comme deuxième argument, nous n'avons plus que 23 caractères.

```
<?php
$chaine = " on étudie le php et etc.... ";
$chaine = trim($chaine);
$chaine = trim($chaine,".");
echo $chaine.'<br>';
echo strlen($chaine);//renvoie 23
?>
```

Recherche d'une sous-chaine en PHP : Fonction strpos et strstr

Il y a deux façons de trouver une sous chaîne. La fonction strpos() retourne un nombre qui indique la position de la première occurrence du caractère recherché. Quant-à la fonction strstr(), elle permet elle aussi de trouver la première occurrence d'un caractère. **Sa seule différence avec strpos() est qu'elle retourne le reste de la chaîne à partir de la chaîne repérée.**
Elles ont 2 deux arguments : la chaine dans laquelle on effectue la recherche et le caractère à rechercher.

```
<?php
$adresse = "nom@domaine.fr";
$arobase = "@";
echo strpos($adresse,$arobase) . "<br />"; //retourne la position :5
echo strstr($adresse,$arobase) . "<br />"; //retourne la chaine après avec la chaine recherchée:
@domaine.fr
?>
```

Les deux fonctions ont chacune leurs équivalent : Les fonctions stripos et stristr pour l'analyse sans prendre en compte la casse, strrpos qui trouve la position de la dernière occurrence d'un caractère dans une chaîne, etc.

Récupération d'une sous-chaine : Fonction substr

Maintenant, on va voir comment récupérer une partie de la chaine de caractère. **On veut par exemple afficher juste les 10 premiers caractères dans un long texte**. La fonction substr nous permet de faire cela. Il faut juste lui indiquer la position de départ et le nombre de caractères à enlever.
Dans notre exemple ci-dessous, la valeur 0 indique que la position départ est le début du texte:

```
<?php
$chaine = "Dans ce texte, essayons d'afficher juste les 10 premiers caractères. On coupe donc le texte.";
echo substr($chaine, 0, 10); // affiche " Dans ce te "
?>
```

Remplacer un motif dans une chaîne : str_replace

Dans les emails que l'on envoie (un mail de confirmation par exemple), on remarque que le contenu est le même avec juste le prénom de la personne qui change. Essayons de faire quelque chose de ce genre avec la fonction str_replace. Comme son nom l'indique, **la fonction str_replace permet de chercher une sous chaine et de la remplacer**.
La syntaxe de cette fonction en PHP est la suivante :

```
<?php
str_replace($cherche, $remplace,$texte),
?>
```

$cherche est la valeur à rechercher, $remplace est la valeur avec laquelle on la remplace et $texte est le texte à l'intérieur duquel toute l'opération se déroule. Cette expression est donc équivalent à chercher tous les $cherche dans $texte et les remplacer par $remplace.
Exemple : Cherchons le mot "personne" et remplaçons le par le prénom "Marcel" par exemple.

```
<?php
$chaine ="Cher personne, <br />Nous vous remercions d'avoir bien voulus nous rejoindre. Merci personne de
votre participation.C'est très aimable de votre part.<br />A bientôt, personne";
echo str_replace( "personne", "Marcel", $chaine);
?>
```

Changement de casse : strtoupper et strtolower

Il s'agit maintenant de transformer les minuscules en majuscule ou inversement. Les fonctions en question sont strtolower pour avoir des textes en minuscules et strtoupper pour avoir des textes en majuscule.
Généralement, elles sont utilisées juste avant l'affichage pour normaliser la présentation des données. Dans un formulaire par exemple, s'il est nécessaire de mettre un nom en capitale, on n'est plus obligé d'indiquer à l'internaute de saisir son nom en majuscule. On peut le laisser faire comme bon lui semble et faire le traitement dans le code pour changer.

```
<?php
$nom = "Vernes";
$mail = "JULES@domaine.fr";
echo $nom. " devient " . strtoupper($nom) . "<br />"; //Mettre en majuscule
echo $mail . " devient " . strtolower($mail) . "<br />";//Mettre en minuscule
?>
```

Fonctions PHP : ucfirst et lcfirst

Toujours dans le même contexte, les fonction ucfirst() et lcfirst() convertissent en majuscules respectivement le premier caractère de la chaîne et le premier caractère de chaque mot.

```
<?php
$phrase = "une phrase débute toujours par un majuscule.";
echo $phrase . "<br />";
echo ucfirst($phrase) . "<br />";

$acronyme = "fond monétaire internationnale";
echo $acronyme . "<br />";
echo ucwords($acronyme);
?>
```

Les tableaux en PHP

PHP propose une autre façon de grouper et de manipuler les données: les tableaux. Il y a deux types de tableaux: les tableaux à index numériques et les tableaux associatifs. Ces deux types de tableau ont chacun leur syntaxe de déclaration.

Les tableaux à index numériques : array

Il s'agit d'une simple liste d'éléments. Pour les créer, on emploi le mot clé array(), et on sépare les élément par des virgules. Dans la liste, chaque élément est repéré par un numéro unique. Ce numéro est appelé index et est attribué automatiquement suivant les ordres.
Pour avoir accès à la valeur de chaque élément, on met la variable tableaux suivi de l'indice de l'élément mis entre crochet. Lors de la déclaration d'un tableau, il n'est pas nécessaire d'indiquer leur taille. Ceci est géré par PHP. Dans un tableau indexé numériquement le premier indice est par défaut 0.

```
<?php
$etudiant = array("Larah", "Sebastian", "Kevin", "Audile", "Rodrigue");
echo $etudiant[0].'<br>';
echo $etudiant[1].'<br>';
echo $etudiant[2].'<br>';
echo $etudiant[3].'<br>';
echo $etudiant[4].'<br>';
?>
```

Boucle, Tableau et fonction count

La fonction count permet d'avoir le nombre d'éléments que contient un tableau. En faisant un peu de **révision sur les boucles**, on peut avoir un code plus souple à partir du même exemple :

```
<?php
$etudiant = array("Larah", "Sebastian", "Kevin", "Audile", "Rodrigue");
for($i = 0;$i <= count($etudiant); $i++)
{
  echo $etudiant[$i] . "<br />";
}
?>
```

Les tableaux associatifs en PHP

Lors de la définition d'un tableau associatif, on indique nous même les indices du tableau. Dans la mesure où l'on est libre de les spécifier nous même, les indices peuvent être non seulement des nombres mais aussi des textes. Dans ce cas on parle de 'clé'. Ce type de tableau est très pratique dans le cas où l'on s'intéresse à la signification des valeurs contenues.
Toujours à l'aide du mot clé array, la définition d'un tableau associatif a la syntaxe suivante :

```
$tableau= array(
cle1=>valeur1 ,
cle2=>valeur2 ,
...);
```

On sépare donc les différentes clés valeur par des virgules. **Pour accéder à un élément du tableau, on peut faire comme précédemment en indiquant la valeur de la clé comme indice**. Voyons tout cela dans un exemple plus concret :

```
<?php
$personne = array(
"prenom" => "Jessy",
"nom" => "Brown",
"telephone" => "000011111",
);

//affichage de l'élément nom
echo $personne['nom'];
?>
```

Afficher tous les éléments d'un tableau : foreach

Si on veut afficher tous les éléments du tableau, en affichant chaque clé et chaque valeur, on peut utiliser l'instruction foreach comme ceci :

```
<?php
$personne = array(
"prenom" => "Jessy",
"nom" => "Brown",
"telephone" => "000011111", );

foreach ($personne as $key => $value) {
  echo "Clé: ". $key . ", Valeur : ". $value . "<br/ >\n";
```

```
}
?>
```

PHP et tableaux multidimensionnels

Dans notre exemple, on a eu à gérer un tableau avec une seule personne. Vous imaginez peut être un tableau de plusieurs personnes, dont chaque personne a son prénom, nom et téléphone. Une bonne idée mais comment faire ?
Pour cela, **il faut faire ce que l'appelle un tableau multidimensionnel**. Pour faire plus simple, nous nous limiterons aux tableaux à deux dimensions sachant qu'on peut en faire à plusieurs dimensions. Définissons alors des personnes avec chacun leur numéro, nom , prenom et telephone. Après, on peut accéder à leur valeur individuellement (par exemple on veut juste avoir le prénom de la première personne).

```
<?php
$personnes = array(
1 => array('prenom' => 'Jessy', 'nom' => 'Brown', 'telephone' => '00001111'),
2 => array('prenom' => 'Sharon', 'nom' => 'Dain', 'telephone' => '00221111'),
3 => array('prenom' => 'Marta', 'nom' => 'Blanca', 'telephone' => '003311111')
);
echo $personnes[1]['prenom']; //Jessy
?>
```

En utilisant l'instruction foreach , on aura :

```
<?php
$personnes = array(
1 => array('prenom' => 'Jessy', 'nom' => 'Brown', 'telephone' => '00001111'),
2 => array('prenom' => 'Sharon', 'nom' => 'Dain', 'telephone' => '00221111'),
3 => array('prenom' => 'Marta', 'nom' => 'Blanca', 'telephone' => '003311111')
);

foreach($personnes as $cle1 => $valeur1)
{
 echo "personne n°:" . $cle1 . "<br />";

   foreach ($valeur1 as $cle2=>$valeur2)

   {
    echo "Clé : ".$cle2 ."", Valeur: " . $valeur2 . "<br />\n";
   }
}
?>
```

Manipuler les tableaux PHP avec des fonctions

PHP propose de nombreuses fonctions très puissantes pour manipuler des tableaux.
Comme vous allez le voir, il est très facile de trier un tableau ou de compter ses élements.

Connaitre la taille d'un tableau : count

On entend par taille d'un tableau le nombre d'éléments que contient le tableau. Ceci est d'un intérêt multiple dont le principal est au niveau des compteurs de boucle. Dans la mesure où l'on désire utiliser une boucle pour l'affichage d'un tableau, il est nécessaire de connaitre la taille d'un tableau pour spécifier l'arrêt du compteur.
La fonction PHP count() renvoie la taille du tableau qu'on lui passe en paramètre :

```
<?php
$tab[0]=1;
$tab[1]=2;
$tab[2]=3;
$tab[3]=4;
$tab[4]=5;
$tab[5]=6;
$taille =count($tab);
echo 'La taille du tableau est :'.$taille;
?>
```

Recherche d'un élément dans un tableau : in_array

La fonction in_array() indique si une valeur appartient à un tableau. Cette fonction nous informe de la présence d'un élément dans le tableau en nous renvoyant TRUE s'il l'a trouvé et FALSE dans le

cas contraire. Elle a en premier argument l'élément recherché, et en deuxième argument le tableau en question.

```
<?php
$etudiant = array ("Alissa", "Marianne", "Mickael", "Shania", "Odile", "Stefanie");
$recherche = "Marianne";
if(in_array($recherche, $etudiant))
{
echo $recherche." a été trouvé dans la liste des étudiants";
}
else
{
echo $recherche." n'est pas dans la liste des étudiants";
}
?>
```

Si la fonction in_array() permet de vérifier la présence d'un élément dans le tableau, une autre fonction nous permet d'avoir la clé correspondante de l'élément recherché. Il s'agit de la fonction array_search() qui fonctionne de manière similaire à in_array().
La différence est qu'elle retourne la clé correspondante de l'élément qu'elle a trouvé et FALSE si l'élément n'a pas été trouvé.

```
<?php
$etudiant = array ("Alissa", "Marianne", "Mickael", "Shania", "Odile", "Stefanie");
$recherche = "Marianne";
$cle = array_search($recherche, $etudiant);
if($cle === FALSE)
{
  echo $recherche." n'est pas dans la liste des étudiants";
}
else
{
  echo $recherche." a été trouvé dans la liste des étudiants. Son rang est : ".$cle ;
}
?>
```

Nombre d'occurrences d'un élément et gestion des doublons : array_count_value

Le nombre d'occurrences d'un élément d'un tableau est donné par la fonction array_count_value(). Elle renvoie pour chaque valeur le nombre d'occurrences trouvé dans un tableau associatif. Prenons toujours l'exemple des étudiants, en comptant le nombre de fois où Marianne apparait dans le tableau.

```
<?php
$etudiant = array ("Alissa", "Marianne", "Mickael", "Shania", "Odile", "Stefanie", "Marianne");
$occurence = array_count_values($etudiant);
echo "L'étudiant Marianne apparait ". $occurence['Marianne']." fois dans le tableau";
?>
```

Nous avons vu que Marianne apparait deux fois dans le tableau. Dans le cas où l'on a besoin de supprimer les doublons, la fonction array_unique () est à notre disposition. Elle retourne un nouveau tableau sans doublons quand on lui passe en paramètre le premier tableau.

```
<?php
$etudiant = array ("Alissa", "Marianne", "Mickael", "Shania", "Odile", "Stefanie", "Marianne");
//tableau avant :avec doublons
echo "avec doublons : ";
echo "<pre?>";
print_r($etudiant);
echo "</pre?>";

//enlever les doublons
$etudiant= array_unique($etudiant);

echo "sans doublons : ";
//tableau après : sans doublons
echo "<pre?>";
print_r($etudiant);
```

```
echo "</pre?>";
?>
```

Trier les tableaux avec PHP

Comme nous allons le voir, il est possible de trier un tableau de différente manière. Là encore, la souplesse de PHP va nous simplifier la tache.

Tri d'un tableau par valeur

La manière la plus simple et la plus utilisée pour trier les tableaux est de les trier par valeurs. Si l'on veut avoir par exemple un tableau des étudiants par ordre alphabétique, la fonction sort() permet de faire le tri. L'argument fourni sera le tableau lui-même.

```
<?php
//tri
$etudiant = array ("Alissa", "Mickael", "Shania", "Odile", "Stefanie", "Marianne");
echo "avant tri : ";
echo "<pre>";
print_r($etudiant);
echo "</pre>";

//trier le tableau
sort($etudiant);

//après tri
echo "après tri : ";
echo "<pre>";
print_r($etudiant);
echo "</pre>";
?>
```

Remarques sur les tris

Quelle est la différence entre :

```
1
2
3
4
11
12
13
```

... et ...

```
1
11
12
13
2
3
4
... ?
```

C'est ce que nous allons voir ci-dessous.

Les clés d'un tableau : SORT_NUMERIC, SORT_STRING

Pour forcer une comparaison numérique, il faut fournir SORT_NUMERIC en second paramètre de la fonction sort(). Il en est de même pour la comparaison textuelle : SORT_STRING.
Il est aussi possible de faire des tris en ordre inverse, c'est-à-dire un tri décroissant. Cela est possible grâce à la fonction rsort().

Fonctions de tri : ksort, krsort, natsort et usort

Il existe d'autre fonctions nous fournissent une variété très complète de façons de trier un tableau :

- tri par clé : ksort() et krsort
- tri naturel : natsort()
- tri avec une fonction utilisateur : usort()

Je vous laisse le soin de les découvrir et de les employer selon vos besoins

Taiter et manipuler les tableaux avec PHP

PHP nous fournit encore un bon nombre de manière pour traiter et manipuler les tableaux. Cette fois ci, on se penchera dans des études plus poussées sur les tableaux, notamment sur la relation entre chaîne de caractère et les tableaux, opération entre plusieurs tableaux, etc.
Bref, on abordera plusieurs techniques usuelles sur l'utilisation des tableaux.
Une relation étroite existe entre les traitements de chaînes de caractères et les tableaux.
On peut alors transformer une séquence de chaîne de caractère en tableau, et inversement un tableau en une séquence de chaines de caractères.
Un séparateur jouera le rôle d'identificateur du découpage et mettra en exergue chaque élément du tableau. Voyons tout de suite cela à l'aide d'un exemple :

```
<?php
$list_fruit = "pomme, orange, kiwi, poire";
$tab_fruit = explode(",", $list_fruit);
echo "Le tableau des fruits:";
echo "<pre>";
print_r($tab_fruit);
echo "</pre>";
?>
```

Ici, la fonction explode() crée un tableau avec la chaine de caractère et le tableau crée contient comme élément chaque sous chaine délimitée par le séparateur : ",".
Inversement, **un autre principe consiste à concaténer tous les éléments du tableau de façon à ce que le séparateur en argument sépare les éléments en plusieurs groupes distincts**. Le résultat sera alors une chaine de caractères composée des éléments du tableau dont chaque élément du tableau est délimité par le séparateur.
Cela nous permet de générer une liste par exemple.

```
<?php
$tab_fruit = array("pomme", "orange", "poire");
$list_fruit = implode($tab_fruit, ",");
echo $list_fruit;
?>
```

Division et fusion de tableaux avec PHP

Il est souvent utile de diviser un tableau en plusieurs sous tableaux ou bien encore de fusionner plusieurs tableaux afin d'en obtenir un de taille plus conséquente. Voyons voir comment faire cela.

Division d'un tableau en plusieurs : array_chunk

"Diviser pour mieux régner", parfois il est aussi nécessaire de découper les tableaux en petit tableaux. La fonction array_chunk() sépare un tableau en plusieurs dont la taille est déterminée par le deuxième argument de la fonction.
Découpons par exemple notre fameux tableau des étudiants pour former le tableau des binômes (taille = 2).

```
<?php
$etudiant= array ("Alissa", "Mickael", "Shania", "Odile", "Stefanie", "Marianne", "Sophie","Marco");
echo "Le tableau des étudiants:";
echo "<pre>";
print_r($etudiant);
echo "</pre>";

//Découpage du tableau
$binome = array_chunk($etudiant, 2);

echo "Le tableaux des binômes:";
echo "<pre>";
print_r($binome);
echo "</pre>";

echo "Le premier binôme:";
echo "<pre>";
print_r($binome[0]);
echo "</pre>";
?>
```

Les étudiants sont alors groupés deux à deux et le premier binôme est constitué d'Alissa et de Mickael.

Fusion des tableaux en PHP : array_merge

Cependant "l'union fait la force", une fonction inverse de array_chunk() est là pour ça. Il s'agit de array_merge() qui fusionne les petits tableaux en un grand tableau. Dans notre exemple, les deux premiers binômes d'étudiant veulent s'unir pour former un groupe de quatre étudiants :

```
<?php
//tableau de départ

$etudiant= array ("Alissa", "Mickael", "Shania", "Odile", "Stefanie", "Marianne", "Sophie","Marco");
echo "Le tableau des étudiants:";
echo "<pre>";
print_r($etudiant);
echo "</pre>";

//Découpage du tableau
$binome = array_chunk($etudiant,2);

//Les binômes
echo "Le tableaux des binômes:";
echo "<pre>";
print_r($binome);
echo "</pre>";

//regroupement des deux premiers binômes
$etudiant_regroupe = array_merge($binome[0],$binome[1]);
echo "Le regroupement des deux binôme:";
echo "<pre>";
print_r($etudiant_regroupe);
echo "</pre>";
?>
```

Différence entre tableaux : array_diff

Supposons maintenant que l'on ait deux tableaux de groupes d'étudiants : $natation qui contient les étudiants qui font de la natation et $danse contient les étudiants qui font de la danse.
On souhaite avoir les étudiants qui ne font que de la natation, c'est-à-dire qui ne font pas de la danse. **Dans ce cas on fait la différence des deux tableaux à l'aide de la fonction array_diff()**.

```
<?php
$natation = array ("Alissa", "Mickael", "Shania", "Odile", "Stefanie","Anaia", "Marianne",
"Sophie","Marco","Zora");
$danse = array ("Natacha", "Mickael", "Shania", "Michèle", "Stefanie", "Adrien", "Patrick","Marco");
$diff = array_diff($natation,$danse );
echo "Ces étudiants ne font pas de la danse mais seulement de la natation:";
echo "<pre>";
print_r($diff);
echo "</pre>";
?>
```

Ceci renvoie à l'écran le tableau de la liste des étudiants qui ne font de la natation :

```
Array
(
[0] => Alissa
[3] => Odile
[5] => Anaia
[6] => Marianne
[7] => Sophie
[9] => Zora
)
```

Intersection entre tableaux et fonction array_intersect

A partir du même exemple, construisons maintenant un autre tableau qui contient les étudiants qui font les deux activités à la fois. Autrement dit, cherchons l'intersection entre les tableaux $natation et $danse en utilisant la fonction array_intersect().

```
<?php
$natation = array ("Alissa", "Mickael", "Shania", "Odile", "Stefanie","Anaia", "Marianne",
```

```
"Sophie","Marco","Zora");
$danse = array ("Natacha", "Mickael", "Shania", "Michèle", "Stefanie", "Adrien", "Patrick","Marco");
$intersect = array_intersect($natation,$danse );

echo "Ces étudiants font à la fois de la danse et de la natation:";
echo "<pre>";
print_r($intersect);
echo "</pre>";
?>
```

On aura donc un tableau des étudiants qui font à la fois de la danse et de la natation :

```
Array
(
[1] => Mickael
[2] => Shania
[4] => Stefanie
[8] => Marco
)
```

Comment inclure les fichiers en PHP ?

Dans une programmation en PHP, sans l'inclusion de fichiers on serait obligé de copier coller le même code sur toutes les pages qui nécessitent une instruction identique.
Comme pour les fonctions par exemple, **il se peut que vous ayez à utiliser les mêmes fonctions dans des fichiers différents**. Vous auriez donc à recopier la définition de chaque fonction en haut de chaque page. Cette méthode rendrait les mises à jour plus difficiles. En effet, ça vous obligerait à parcourir toutes les pages pour modifier les définitions des fonctions.
Ce serait très malheureux de perdre du temps à faire ça alors qu'**on a la possibilité de l'éviter en centralisant tout dans un même fichier**.

Inclure un fichier avec la fonction include

Supposons que l'on a quelques fonctions sur le calcul des prix. On en aura souvent besoin dans les pages de notre site. On va les regrouper dans un fichier prix.php et on va essayer de l'inclure dans un fichier. Afin d'éviter de trimbaler ces fonctions partout, **la fonction include nous servira d'inclusion de fichier**.
La fonction en question se remplace elle-même par le contenu du fichier spécifié.
Contenu de prix.php :

```
<?php
function difference($prix1, $prix2)
{
$resultat=$prix1-$prix2;
return $resultat;
}

function calculttc($prixht,$tva)
{
$prixttc=$prixht*(1+$tva/100);
}

function calculht($prixttc,$tva)
{
$prixht=($prixttc*100)/($tva+100);
}

function calculremise($pourcentage, $prix)
{
$remise=$prix*$pourcentage;
}
?>
```

Dans une page catalogue.php, on a par exemple le prix hors taxe au départ. Cependant, on a besoin d'afficher le prix TTC.
Contenu de catalogue.php :

```
<?php
//affichage et calcul prix TTC
include ("prix.php"); //inclusion du fichier prix
$prixht=100;
```

```
define(TVA, 19.6);
$tva = 19.6;
echo "prix ttc:".calculttc($prixht,$tva);

echo calculttc($prixht,$tva);
?>
```

Dans une autre page, facture.php par exemple, on a besoin des différents prix et de calculer aussi la remise.

Contenu de facture.php :

```
<?php
//affichage et calcul de tous les prix
include ("prix.php");
$prixht=100;
define(TVA, 19.6);
$tva = 19.6;
$prixttc= calculttc($prixht,TVA);
$remise = calculremise(10,$prixht);
$apayer= difference($prixttc, $remise);

echo "prix ht:".$prixht."<br>";
echo "prix ttc:".$prixttc."<br>";
echo "prix remise 10%:".$remise."<br>";
echo "prix à payer:".$apayer."<br>";
?>
```

On 'inclue' le contenu du fichier prix.php dans les autres fichiers, en l'appelant par la fonction 'include', et en lui passant en paramètre le nom du fichier à include (dans l'exemple: prix.php).

Inclure des fichiers avec la fonction Require

L'instruction require fonctionne de la même façon qu'include. Toutefois, il existe une différence entre ces deux méthodes. Lorsque vous tentez d'inclure un fichier avec include, si le fichier n'existe pas, PHP affichera une alerte (warning).

Cela veut dire qu'il va vous signaler tout simplement l'erreur mais il continuera l'exécution du script. A l'inverse, la fonction require génèrera une erreur fatale. Il interrompe votre script en cas d'erreur. En d'autres termes, utilisez require() si vous voulez que votre script soit interrompu en cas d'inclusion manquant.

Exemple : Mélanger XHTML et PHP

On a parlé de page web, catalogue, etc mais jusqu'à présent nous n'avons pas trop abordé un sujet important :

Comment mélanger du PHP avec du html ? Voici venu ce moment propice que tout le monde a attendu. **En fait, il est bel et bien possible de mettre du XHTML dans du PHP**.

C'est d'ailleurs le but ultime de l'application de PHP dans le domaine de web. Pour cette expérience, utilisons 3 fichiers:

Contenu du fichier haut.php :

```
<table width="500" border="1" bgcolor="#66CCCC">
 <tr>
  <td><?php echo " Menu1"; ?></td>
  <td><?php echo " Menu2"; ?></td>
  <td><?php echo " Menu3"; ?></td>
  <td><?php echo " Menu4"; ?></td>
  <td><?php echo " Menu5"; ?></td>
  <td><?php echo " Menu6"; ?></td>
 </tr>
</table>
```

Contenu du fichier bas.php :

```
<table width="500" border="1" bgcolor="#FF9900">

  <tr>
 <td><?php echo "Pieds de page"; ?></td>
  </tr>
</table>
```

Contenu du fichier page1.php :

```
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>page</title>
</head>
<?php require("haut.php"); ?>
<table width="500" height="500" border="1" >

 <tr align="center">
<td>

  <?php echo "contenu php"; ?>
 </td>
 </tr>

</table>
<?php require("bas.php"); ?>
</html>
```

Bien sûr, notre page a encore besoin d'une retouche. Néanmoins, la base est déjà là. Ce qui est intéressant c'est que là encore on a bien vu l'intérêt des inclusions de fichier.
En effet, le haut et le bas de page ne changent pas lors de la navigation. Toutes les pages n'ont peut être pas les mêmes contenus, mais le haut et le bas de page restent les mêmes quelque soit la page.
Dans ce cas, il suffit d'inclure le fichier haut.php, et le fichier bas.php dans toutes les pages. S'il y a quelque chose à modifier dans le fichier haut.php, on aura juste à toucher un seul fichier, et ceci impactera l'ensemble des pages faisant l'include.
Par conséquent, structurez votre page ainsi vous évitera certains problèmes.

Inclusion avec les fonctions require_once et include_once

Notez bien qu'il y a aussi deux autres fonctions require_once et include_once qui peuvent vous être utiles.
Ce sont les même que require et include mais la différence est qu'elles s'assurent que le fichier qu'on inclue ne l'a pas déjà. **Il est conseillé de les utiliser lorsque vous voulez être sûrs que le fichier ne sera inclus qu'une seule fois**, notamment pour éviter les déclarations qui engendrent des erreurs.
Un fichier rempli de fonctions, par exemple, ne peut être inclu qu'une seule fois (par exécution). Les fonctions étant déjà déclarées, elle ne peuvent l'être une seconde fois, et cela génèrerait une erreur.

Les tableaux super globaux de PHP

Maintenant que l'on a parlé des tableaux en détail, il est possible de s'intéresser aux tableaux super globaux de PHP.
PHP fourni un grand nombre de variables prédéfinies. En voici une liste tirée du manuel PHP qui va probablement vous servir :
$GLOBALS : Contient une référence sur chaque variable qui est disponible dans l'environnement d'exécution global. Les clés de ce tableau sont les noms des variables globales.
$_GET :C'est le tableau des variables fournies par le protocole HTTP en méthode GET. $_GET a remplacé $HTTP_GET_VARS.
$_POST : C'est le tableau des variables fournies par le protocole HTTP en méthode POST. C'est la nouvelle version de l'ancienne variable $HTTP_POST_VARS qui est maintenant obsolète, mais toujours là.
$_COOKIE : C'est le tableau des variables fournies par le protocole HTTP, dans les cookies. C'est la nouvelle version de l'ancienne variable $HTTP_COOKIE_VARS.
$_FILES :C'est le variables fournies par le protocole HTTP, suite à un téléchargement de fichier. C'est la nouvelle version de l'ancienne variable $HTTP_POST_FILES.
$_ENV : Les variables fournies par l'environnement. C'est la nouvelle version de l'ancienne variable $HTTP_ENV_VARS.
$_REQUEST : Les variables fournies au script par n'importe quel mécanisme d'entrée et qui ne doit recevoir une confiance limitée.
$_SESSION : Les variables qui sont actuellement enregistrées dans la session attachée au script. C'est la nouvelle version de l'ancienne variable $HTTP_SESSION_VARS.

Afficher les éléments d'un tableau super globaux

Pour afficher les éléments de chaque tableau, vous pouvez utiliser la fonction print_r en mettant le tableau en paramètre comme dans l'exemple suivant.

```php
<?php
echo "<pre>";
print_r($GLOBALS); //affiche le nom des super globaux
echo "</pre>";
?>
```

La variable d'environnement $_SERVER

$_SERVER est un tableau des variables fournies par le serveur web, ou bien directement liées à l'environnement d'exécution du script courant. C'est la nouvelle version de l'ancienne variable $HTTP_SERVER_VARS.

Le tableau $_SERVER

Variable	Utilité	Affichage
$_SERVER['REQUEST_METHOD']	La méthode d'appel	POST
$_SERVER['SERVER_NAME']	Nom du serveur	localhost
$_SERVER['SERVER_ADMIN']	L'email de l'administrateur du serveur	email@domaine.tld
$_SERVER['SERVER_ADDR']	L'Adresse IP du serveur	195.14.0.256
$_SERVER['QUERY_STRING']	Les paramètres indiquées à votre script	url=toto.html&id=234
$_SERVER['REMOTE_PORT']	Port HTTP de la requête	80
$_SERVER['REMOTE_ADDR']	Adresse IP de l'internaute	88.101.2.255
$_SERVER['REQUEST_URI']	Chemin du script	/exemple.php
$_SERVER['PATH_TRANSLATED']	Chemin physique (complet) du script	/home/www/domain.fr/example.php
$_SERVER['HTTP_USER_AGENT']	User agent du navigateur du client	Mozilla/5.0 (Windows; U; Windows NT 5.1; fr; rv:1.8.1) Gecko/20061010 Firefox/2.0
$_SERVER['HTTP_REFERER']	L'URL de la page d'où	http://oseox.fr/exemple.php

	provient l'internaute	
$_SERVER['HTTP_HOST']	Le nom de domaine où est exécuté le script	example.fr
$_SERVER['HTTP_ACCEPT_LANGUAGE']	Langue acceptée par le navigateur de l'internaute	fr
$_SERVER['DOCUMENT_ROOT']	Adresse de la racine du serveur	/var/www/exemple.fr/

Fonctions PHP de manipulation de fichiers

Les fichiers sont des éléments sur lesquels travaille l'application. Ils peuvent servir de stockage définitif de données simples à la place de la base de données, mais aussi de stockage de données temporaires.
C'est pourquoi ils méritent une attention particulière. Nous aborderons alors une notion un peu plus avancée sur la gestion, l'utilisation et la manipulation des fichiers.

Tester l'existence d'un fichier : file_exist

Une tentative de manipulation sur un fichier non existant est interprétée comme étant une erreur. C'est pourquoi avant toute manipulation d'un fichier il est toujours nécessaire de vérifier son existence.
La fonction file_exist renvoie true si le fichier existe et false dans le cas contraire. On l'utilise souvent dans une structure alternative comme suit :

```
<?php
if(file_exists("fichier.txt")); { echo "Le fichier fichier.txt existe"; }else { echo "Le fichier n'existe pas"; }
?>
```

Création et effacement : fonctions fopen, fclose, touch et unlink

Si le fichier n'existe pas, il faut alors le créer. Pour cela, il existe deux méthodes. La première consiste à ouvrir le fichier en écriture avec la fonction fopen() et le fermer après. Rappelons que fclose() est la fonction utilisée pour la fermeture d'un fichier.
Si le fichier passé en argument n'existe pas encore il le crée automatiquement.
Exemple

```
<?php
$fp = fopen("fichier.txt", 'w'); //ouverture
fclose($fp); //fermeture
?>
```

Si non, il existe aussi une autre méthode en PHP qui requiert l'utilisation de la fonction touch().Elle permet de créer le fichier s'il n'existe pas ou de mettre à jour ses dates d'accès et de modification dans le cas contraire.
Exemple

```
<?php
if(!file_exists("fichier.txt"));
touch("fichier.txt");
?>
```

A l'inverse, la fonction unlink() supprime directement le fichier.

```
<?php
if(!file_exists("fichier.txt"));
unlink("fichier.txt");
?>
```

Copie et déplacement de fichiers : copy et rename

La fonction utilisée pour le déplacement d'un fichier est rename(). Celle qui est utilisé pour la copie est copy().
Les deux fonctions fonctionnent de la même manière et ont chacune deux arguments : en premier l'adresse source et en second l'adresse de destination.
Les syntaxes sont donc les suivantes :

```
copy("source", "destination");
rename("dorigine","renome") ;
```

Informations sur le fichier avec PHP

PHP met à notre disposition diverses informations sur le fichier qui peuvent nous être utilise pour manipuler en toute sérénité notre fichier. Il y a juste à faire appel à la fonction correspondante.

Emplacement d'une fichier : realpath

L'emplacement d'un fichier est par défaut relative au répertoire en cours. C'est à dire que si' l'on n'indique pas une adresse absolue (du style /rep/fichier.txt) au moment de l'ouverture, **le programme recherche le fichier en question dans le répertoire courant**.
Lorsqu'il s'agit d'un fichier distant, on accède de la même façon qu'un fichier locale. Seulement, il faut préfixer l'adresse d'un " ftp :// " ou d'un " http:// ".
On peut connaitre le chemin réel, notamment le nom et l'adresse d'un fichier à l'aide de la fonction realpath().
-La taille d'un fichier
La fonction filesize () renvoie la taille du fichier quand on passe en argument le nom de celui-ci.

```
<?php
$fichier = "fichier.txt";
if(file_exists($fichier))
{
echo "La taille de ".$fichier." est de : ".filesize($fichier);
}
else {
echo "Le fichier n'existe pas";
}
?>
```

Dates du fichier filectime et filemtime

La connaissance de la date dans lors de la manipulation de fichier permet de suivre son évolution. La fonction filectime() nous donne la date de création du fichier et la fonction filemtime() nous permet de savoir la date de sa dernière modification.
Exemple

```
<?php
$fichier = "fichier.txt";
if(file_exists($fichier))
{

  if(filemtime($fichier)<filectime($fichier))
   {
    echo "Pas de modif depuis la création";
  }
  else
  {
  echo "Le fichier a été modifié depuis la création";
  }

}
else {
  echo "Le fichier n'existe pas";
}
?></filectime($fichier))
```

Les expressions régulières avec PHP

PHP nous offre une variété de fonctions pour le traitement des chaînes de caractères. Parmi elles, on trouve les fonctions telle que strpos() pour rechercher une chaîne de caractères bien définie. Cependant, elles se limitent à des usages simples et il arrive qu'elles ne suffisent pas pour les manipulations de chaînes de caractère que l'on souhaiterait faire.

Dans le cas où l'on veut par exemple trouver les chaines qui commencent par un C, qui contiennent deux ou trois fois le caractère "a" et se terminent par un point, il est plus simple de faire appel aux expressions régulières.
En effet, ces dernières permettent de définir des critères de recherches beaucoup plus complexes. Elles offrent plusieurs possibilités notamment concernant la notion de répétion, la gestion des casses, gestion des nombre d'occurrence, etc. En général, dans une expression régulière on définit un motif pour valider la chaine lors d'une recherche.

Règles de base d'une expression régulière

Avant d'entamer le vif du sujet, notons que dans une expression régulière, une chaine de recherche est entourée de délimiteurs. Ces derniers peuvent être n'importe quel caractère non alphanumérique (ni lettre ni chiffre) sauf l'antislash "\". D'habitude, on utilise le slash "/" comme délimiteurs, mais une paire d'accolades, de crochets ou de parenthèses est aussi autorisée.
Quand à l'antislash "\" , il est utilisé pour échapper les caractères spéciaux. En effet, **certain caractères peuvent avoir une signification particulière dans une expression régulière**.
Parfois certains caractères sont aussi utilisés comme critères de recherche. Dans ce cas, afin d'éviter toute ambigüité, nous allons devoir la protéger en utilisant le caractère d'échappement.
Notons aussi que les règles habituelles des chaînes de caractères s'appliquent également aux expressions régulières. **Ainsi, dans une expression régulière, la chaîne \n fait partie des caractères spéciaux**. Si elle fait partie des critères de recherche, il est nécessaire de la faire précéder d'un antislash.

Chaîne de recherche simple avec preg_match

La fonction preg_match() est le plus souvent utilisée en cas de recherche de caractère.
Elle peut prendre en premier argument une expression régulière et en second une chaîne de caractères référence. Si la chaîne de référence est valide par rapport à l'expression régulière, alors la fonction renvoie la valeur TRUE, si non elle renvoie FALSE.
Quand au troisième argument (ici : $out) qui est un tableau, il est d'une utilité particulière que l'on verra plus tard.

```
<?php
$texte = 'La fille est sage';
if(preg_match_all('/fille/', $texte,$out))
{
echo "Le texte contient la chaine fille";
}
else
{
echo "Le texte ne contient pas la chaine fille";
}
?>
```

La fonction preg_match() est utilisée de manière assez simple dans cet exemple. **A la place de la fonction strpos(), elle va vérifier la présence de la chaîne "fille" dans le texte "La fille est sage"**.
Ceci dit, on trouvera mieux son intérêt dans une utilisation plus complexe, c'est-à-dire en construisant des expressions.

Construction d'expressions régulières

Pour construire une vraie expression régulière, plusieurs notions nécessitent d'être abordées.

Choix entre plusieurs chaines de caractère : preg_match_all

Les expressions régulières permettent de définir des critères de façon très souple. Dans l'exemple ci-dessus, on n'a recherché que le mot fille, mais supposons que l'on désire vérifier si la chaîne contient soit "une" soit "la". **Voire même si la chaîne contient la fille ou une fille**.
Pour cela, la barre verticale (pipe, caractère |) nous servira de délimiteur entre deux alternatives. Autrement dit, le caractère | se comporte comme un opérateur OU dans les expressions régulières.
Dans le code suivant, l'expression une|la trouve les chaînes contenant au moins une des deux chaines.

```
<?php
$texte = 'La fille est sage';
if(preg_match_all('/La|Une/', $texte,$out))
{
echo "Le texte contient l'une des chaines : La ou Une";
}
```

```
else
{
echo "Le texte ne contient aucune des chaines : La ou Une";
}
?>
```

Et si l'on rajoute le mot fille après, on peut trouver les chaîne qui contient "une fille" ou "la fille".

```
<?php

$texte = 'Une fille est sage';
if(preg_match_all('/(La|Une) fille/', $texte,$out))
{
echo "Le texte contient l'une des chaines : La fille ou Une fille";
}
else
{
echo "Le texte ne contient aucune des chaines : La fille ou Une fille";
}
?>
```

Liste de caractères autorisés ou interdits

Si vous devez introduire plusieurs alternatives dans votre critère de recherche, il est possible d'utiliser une syntaxe réduite à la place de plusieurs opérateur "|". **Il s'agit de lister tous les caractères autorisés entre des crochets**.
Par exemple [0123456789] permet de valider n'importe quel chiffre. Dans l'exemple suivant, on vérifie la présence de l'âge dans le texte.

```
<?php
$texte = "La fille de 8 ans est sage";
$recherche = '/[0123456789] ans/';
echo preg_match_all($recherche, $texte,$out);
?>
```

Pour lister des caractères interdits, on raisonne pareil, sauf que l'on doit ajouter un accent circonflexe avant la chaine de caractères. **Cependant, certains confondent la définition d'une classe de caractères par négation à l'exclusion des caractères.**
L'expression /[^b]/ peut valider une chaîne qui contient la lette b. Cela signifie que l'on cherche un caractère autre que b, c'est-à-dire valider les chaînes contenant des caractères autres que b. Il ne s'agit donc pas d'une exclusion des caractères concernés.

Classes de caractères prédéfinis

Il peut également être utile de vérifier si une chaîne contient des caractères d'un certain type (numérique, alphanumérique, ...) sans avoir à les énumérer. **Il existe déjà des alternatives prédéfinis qui permettent de vérifier si une chaîne contient des caractères d'un certain type**.
Ces classes de caractères sont entourées par un crochet et un deux- points dont voici la syntaxe : [:classe:]
Vous verrez dans le tableau suivant une récapitulation de certaines de ces classes :

Memo pour vos expressions régulières	
Nom de la classe	**Description**
[:alnum:]	caractères alphanumériques (équivalent à [A-Za-z0-9])
[:alpha:]	caractères alphabétiques ([A-Za-z])
[:blank:]	caractères blanc (espace, tabulation)
[:ctrl:]	caractères de contrôle

[:digit:]	chiffre ([0-9])
[:graph:]	caractère d'imprimerie (qui fait une marque sur l'écran en quelque sorte)
[:print:]	caractère imprimable (qui passe à l'imprimante ... tout sauf les caractères de contrôle)
[:punct:]	caractère de ponctuation
[:space:]	caractère d'espacement
[:upper:]	caractère majuscule
[:xdigit:]	caractère hexadécimal

Exemple d'utilisation des classes de caractère dans une expression régulière :
"[:punct:]|[:space:]" vérifier si une chaîne contient un caractère de ponctuation ou un caractère d'espacement

Début et fin d'une chaine

Pour désigner le début et la fin d'une chaine, on utilise respectivement les symboles ^ et $.
Ainsi :

- "^start": désigne une chaîne qui commence par le mot "start">
- "end$": désigne une chaîne qui se termine par le mot "end"
- "^ok$": désigne une chaîne qui commence et se termine à la fois par le mot "ok"

Gestion des occurrences

Dans une expression régulière, les symboles + , ? et * permettent de définir un nombre d'occurrence d'un caractère.

- un ou plusieurs : "+"
- "abcd+": valide une chaîne qui contient "abc" suivie de un ou plusieurs "d" ("abcd", "abcdd" ...)
- zéro ou un : "?"
- "abcd?": valide une chaîne qui contient "abc" suivie de zero ou un "d" ("abc" ou "abcd")
- zéro ou plusieurs : "*"
- "abcd*": valide une chaîne qui contient "abc" suivie de zero ou plusieurs "d" ("abc", "abc" ...)
- Il est également possible de faire pareil sur plusieurs caractères en groupant une suite de caractères dans des parenthèses.
- "a(bc)*": chaîne qui contient "a" suivie de zero "bc" ou plus

Nombre d'occurrences fixes

On peut aussi fixer de vrais nombres en utilisant les chiffres correspondants avec des accolades. Ainsi, abcd{3} correspond à 3 fois de répétition de la dernière lettre (ici : "d"). Ce qui est équivalent à abcddd.

Intervalle de nombre d'occurrence

Toujours avec des chiffres et des accolades, il est aussi possible de définir des intervalles. On veut par exemple chercher une chaîne répétée entre 3 et 6 fois. Pour cela, on sépare les deux bout par une virgule : abcd{3,5+ valide abcddd, abcdddd, abcddddd, abcdddddd
Notons que si nous omettons le minimum il sera considérée comme nul et si nous omettons le maximum, il sera considéré comme infini.

Exemple et syntaxe d'une expression régulière

Vous vous êtes demandez pourquoi on apprend les expressions régulières, quel est donc l'utilité ? En regardant l'exemple ci-dessus vous seriez en mesure de répondre à cette question.

Cet exemple a été extrait d'un traitement de texte d'un mail sur un site multilingue. Voici l'extrait du contenu du mail en HTML.

```
##Bonjour##,
##Vous êtes désormais inscrit sur le site##.
##Voici vos codes d'accès## :
##Identifiant##: {email}
##Mot de passe## : {motdepasse}
```

Le code suivant recherche tous les textes entouré d'une "##" dans un texte de mail, et les remplace par les traductions correspondantes tout en enlevant les "##". Il faut relever tous les textes à traduire en excluant les balises HTML et tout la mise en forme. On entoure donc d'une double dièze "##" pour dire que c'est un texte à traduire.
Supposons qu'une fonction traduction() effectue la traduction des mots français en anglais. Ainsi : traduction ("Bonjour") pourra renvoyer "Hello". La variable $mail contient tout le contenu du mail :
Exemple Code 1

```
<?php
while(!($debut== false))
{
 $fin=strpos($mail, "##",$debut+2);
 if (!($fin== false))
 {
  $texte = substr($mail,$debut,$fin-$debut+2);
  $traduction = substr($mail,$debut+2,$fin-$debut-2);
  $traduction = t($traduction);
  $mail = str_replace($texte, $traduction, $mail);
  $debut = strpos($mail, "##", 0);
 }
 else
 {
  $debut=false;
 }
?>
```

Voici une autre version qui fait la même chose mais utilisant les expressions régulières. $out est un tableau renvoyé par la fonction preg_match_all.
Exemple Code 2

```
<?php
preg_match_all ("#[\#\#^\#\#\#\#][\#\#](.*?)[\#\#][\#\#]#i", $mail, $out);
foreach($out[0] as $k => $v)
{
 $texte = t($out[1][$k]);
 $mail = str_replace ($v, $texte,$mail);
}
echo $mail ;
?>
```

En ne comparant que le nombre de lignes de code, on peut déjà voir la grande utilité des expressions régulières.
Par la suite nous vous proposons quelques fonctions de bases pour la manipulation avec des expressions régulières.

Les fonctions ereg et eregi

La fonction ereg() est à utiliser tel que :
Booleen ereg(chaîne modele, chaîne texte[,tableau])
Elle recherche dans la chaîne texte les séquences de caractère qui correspondent au modèle qui est une expression régulière. Le troisième argument est optionnel, mais quand il est défini, il permet de stocker toutes les occurrences trouvées dans un tableau.
On passe alors en troisième argument le nom du tableau en question. Lorsque la fonction trouve des occurrences de chaine de caractères, elle renvoie TRUE, sinon elle retourne FALSE.
La fonction eregi() dont la signature est pareille, effectue le même travail que ereg().La seule différence est le fait qu'elle n'est pas sensible à la casse. Autrement dit, la fonction eregi() ne fait pas de différence entre les minuscules et majuscules.
Exemple
L'exemple suivant prend une date au format ISO (YYYY-MM-DD) et l'affiche sous la forme DD.MM.YYYY. Chaque élément du tableau $regs correspond à une partie de la date.

```
<?php
$date="2008-01-26";
if (ereg ("([0-9]{4})-([0-9]{1,2})-([0-9]{1,2})", $date, $regs))
{
echo "$regs[3].$regs[2].$regs[1]";
}
else
{
echo "Format invalide : $date";
}
?> ?>
```

Les fonctions ereg_replace et eregi_replace

La fonction ereg_replace() permet de faire la même chose que la fonction ereg(), c'est-à-dire trouver des occurrences de chaines de caractère qui correspondent au modèle. Mais en plus elle permet aussi de remplacer les occurrences par une chaîne de remplacement.
Sa syntaxe est la suivante :

- chaîne ereg_replace(chaîne modele,chaîne remplacement,chaîne texte)
- eregi_replace() fonctionne de la même façon sauf qu'il est insensible à la casse

Date et timestamp unix avec PHP

Afin de bien appréhender la manipulation des dates en PHP, il est bien nécessaire de comprendre ce qu'est un timestamp unix.
Appelé aussi horodatage unix, **le timestamp unix représente le nombre de secondes écoulées depuis le 1er janvier 1970 à minuit GMT**. Cette date correspond au début de l'aire linux. En effet, les systèmes Unix enregistrent la date et l'heure courantes sous la forme d'un entier sur 32 bits.
Connaître le timestamp d'un moment donné va permettre de travailler avec pour obtenir toutes sortes d'informations : Vérifier la validité d'une date, classer des membres par ordre d'inscription, etc...

Connaitre le timestamp actuel : fonction time

Pour obtenir le timestamp actuel, il vous suffit d'utiliser la fonction time(). La fonction time() est la fonction qui retourne le temps écoulé depuis ce premier janvier 1970 jusqu'à maintenant.
Exemple d'utilisation de time :

```
<?php
echo "Il s'est écoulé ". time() . " secondes depuis le 1er Janvier 1970";
?>
```

Afficher le timestamp d'une date : fonction mktime

Pour convertir une date quelconque en nombre de secondes depuis le 1er Janvier 1970, on va avoir recours à la fonction mktime(). En tant que paramètre de la fonction on précise, l'heure, la minute, la seconde, le mois, le jour, l'année.
Exemple d'utilisation de mktime :

```
<?php
echo "Noël 2007, c'était il y a ". mktime(0,0,0,12,25,2007) . " secondes ";
?>
```

Afficher l'heure du serveur : fonction date

La fonction date() permet d'obtenir l'heure locale du serveur. Elle donne la date d'aujourd'hui. On peut lui passer comme argument des formats de date. Ainsi, elle va formater la date selon le format indiqué en argument.
Les arguments pouvant être employés sont nombreux mais en voici quelques exemples :

- d - Jour du mois, sur deux chiffres (éventuellement avec un zéros) : "01" à "31"
- D - Jour de la semaine, en trois lettres (et en anglais) : par exemple "Fri" (pour Vendredi)
- F - Mois, textuel, version longue; en anglais, i.e. "January" (pour Janvier)
- h - Heure, au format 12h, "01" à "12"
- H - heure, au format 24h, "00" à "23"
- g - Heure, au format 12h sans les zéros initiaux, "1" à "12"
- G - Heure, au format 24h sans les zéros initiaux, "0" à "23"

La fonction getdate() quant à elle, renvoie un tableau associatif contenant les différentes parties du timestamp qu'on lui passe en paramètre. Si on ne lui passe pas de paramètre, elle retourne la date d'aujourd'hui.
Les champs du tableau sont les suivants :

- "seconds" - secondes, numérique
- "minutes" - minutes, numérique
- "hours" - heures, numérique
- "mday" - jour du mois, numérique
- "wday" - jour de la semaine, numérique. 0: dimanche jusqu'à 6: samedi
- "mon" - mois, numérique
- "year" - année, numérique
- "yday" - jour de l'année, numérique
- "weekday" - jour de la semaine format plein texte; c'est à dire. "Monday"
- "month" - mois, format plein texte; i.e. "April"

Contrôle de validité d'une date : fonction checkdate

La fonction checkdate() nous permet de s'assurer de la validité d'une date issue par exemple d'un formulaire. Elle se révèle particulièrement utile pour vérifier les dates saisies par l'utilisateur.
Ce qui nous permet d'éviter les 31 Février, 35 Décembre et autres erreurs.
La fonction vérifie d'abord si l'année est bien un entier compris entre 0 et 32767. Ensuite, elle vérifie si le mois est un entier compris entre 1 et 12. Et enfin, si le jour existe bien dans le mois en question. Sachant que la fonction tient compte des années bissextiles.

```
<?php
$mois = 12;
$jour = 35;
$annee = 2001;
if (checkdate($mois, $jour, $annee)) {
  echo "La date $jour/$mois/$annee est valide";
} else {
  echo "La date $jour/$mois/$annee est invalide";
}
?>
```

Affichage d'une date selon un format donné : fonction strftime

Pour afficher une date, on pourra utiliser la fonction strftime avec pour paramètre une chaîne de caractères indiquant le format sous lequel vous voulez représenter la date.

```
<?php
echo "Affichage au format jour/mois/annee heure:minute:seconde ".
strftime("%d/%m/%y %H:%M:%S")."<br />";
?>
```

Il est également possible de récupérer la date à la microseconde près en utilisant la fonction gettimeofday().
Exemple d'utilisation de gettimeofday

```
<?php
$tableau = gettimeofday();
echo "Il s'est écoulé ".$tableau['sec']." secondes et ".
$tableau['usec']." microsecondes depuis le 1er Janvier 1970.<br />";
?>
```

La fonction date() permet d'obtenir une date plus classique telle que nous la connaissons. La syntaxe est date (format [, timestamp])
Exemple 1 :

```
<?php
echo date ("d/m/Y H:i:s", 1207742661)
?>
```

Exemple 2 :

```
<?php
echo date (" d/m/Y", str(" 2008-10-09) ; /* affichage en format ISO/US d'un format français */
?>
```

Exemple 3 :

```
<?php
setlocale(LC_TIME, 'fr, FR','fr_FR@euro','fr','FR','fra_fra','fra');
echo "nous sommes le". strftime ("%A %d %B %Y et il est %Hh%M", 1207742661); /*affiche la date dans le local actuel */
?>
```

Exemple 4 :

```
<?php
$time=mktime(date('H'),date('i',date('s'),date('m')-8,date('d')-29,date('y')-5) ;
echo date(("d/m/Y H:i:s", $time);/*calcul de la d'il y a 5 ans et 8 mois et 29 jours*/
?>
```

Remarques :

- strftime() se décline en gmstrftime() qui affiche la date GMT à partir d'une date définie localement.
- La fonction date() a à peu près le même role que strftime() mais est moins complète et ne permet pas de clairement différencier le texte, proprement dit, des caractères "clés". Toutefois, les caractères "clés" diffèrent.
- date() se décline également en gmdate() qui affiche la date GMT à partir d'une date définie localement.

Enregistrer des données dans un fichier avec PHP

Nous avons appris à envoyer des données via des formulaires. Il nous vient maintenant à l'esprit de vouloir stocker ces données quelque part. Pour cela PHP nous propose plusieurs choix sur la façon de stocker les données : Entre autres, il est possible de les stocker dans des bases de données ou dans des fichiers.
L'accès aux fichiers locaux est rapide. **C'est donc un avantage non négligeable d'utiliser les fichiers comme support de données**. Toutefois, tout dépend de ce que vous voudrez faire avec les fichiers plus tard. Si vous avez peu de traitements et de tris à effectuer sur le contenu, l'utilisation des fichiers est plus intéressante que celle des bases de données.
PHP propose bon nombre de fonction pour la manipulation des fichiers, qui va de son ouverture à l'écriture dans un fichier texte, ou de la lecture d'un fichier texte jusqu'à sa fermeture. Avec toutes les richesses syntaxiques du langage, **on peut même traiter des fichiers autres que les simples fichiers textes**. Fichiers csv, par exemple.

Ouverture d'un fichier : fopen

La fonction fopen
La fonction PHP de base pour l'ouverture d'un fichier est la fonction fopen(). Sa syntaxe est la suivante :

```
fopen(fichier, mode);
```

Le premier argument de la fonction open peut être le nom du fichier lui-même ou un chemin d'accès vers le fichier lui-même :
Par exemple : /www/site/fichier.txt, ou fichier.txt
Mode d'ouverture
Le mode d'ouverture définit spécialement les accès au fichier. Le système d'exploitation a besoin de connaitre l'usage que vous voulez faire du fichier une fois qu'il sera ouvert. C'est à partir de là qu'il peut spécifier les accès : en mode écriture, lecture, etc. Autrement dit, ça lui permet de savoir :

- D'une part si vous avez les autorisations requises pour manipuler le fichier sous cette mode
- D'autre part si le fichier pourra être ouvert par un autre script une fois que vous l'aurez ouvert

Pour indiquer le mode d'ouverture du fichier on emploi une lettre et on le met en deuxième argument de la fonction fopen.
On a donc plusieurs possibilités :
On peut ouvrir le fichier en lecture seule, en écriture seule ou bien encore en lecture et en écriture. Pour écrire dans le fichier on peut soit écraser le contenu du fichier par les nouvelles données (c'est-à-dire tout effacer et tout réécrire), soit garder le contenu existant et y ajouter les nouvelles données à la suite.
En résumé, voici la liste des modes possibles avec leurs significations :

- **r** : Ouvre un fichier en lecture seule, place le pointeur en début de fichier (r pour read).
- **r+** : Ouvre le fichier en lecture et écriture, place le pointeur en début de fichier.
- **w** : Ouvre un fichier en écriture seule, place le pointeur en début de fichier. Si le fichier existe déjà, son contenu est écrasé, dans le cas contraire il crée le fichier (w pour write).
- **w+** : Ouvre le fichier en mode lecture et écriture, place le pointeur au début du fichier. Si le fichier existe déjà, son contenu est écrasé, dans le cas contraire, il crée le fichier.
- **a** : Ouvre le fichier en écriture seule, place le pointeur à la fin du fichier. Si le fichier n'existe pas, on tente de le créer (a pour append).
- **a+** : Ouvre le fichier en mode lecture et en écriture, place le pointeur à la fin du fichier. Si le fichier n'existe pas, on tente de le créer.

Ecriture dans un fichier : fwrite

La fonction fwrite

La fonction utilisée pour l'écriture dans un fichier est la fonction fwrite().
Selon le prototype suivant, elle peut posséder 3 arguments :

```
int fwrite ( resource fichier, string chaine [, int longueur])
```

Elle retourne une valeur de type int qui correspond au nombre de bytes écrites ou FALSE en cas d'erreur. Le troisième paramètre qui est facultatif est la longueur. Lorsqu'il est spécifié, il indique le nombre maximal d'octets à écrire.
Voici un exemple d'utilisation de fwrite, avec ses deux arguments. Il tentera de créer le fichier :

```
<?php
$fp=fopen("fichier.txt", "w"); //ouverture du fichier en mode écriture, création du fichier s'il n'existe pas.
fwrite($fp,"Un texte dans votre fichier"); // insert le texte: Un texte dans votre fichier.
?>
```

Reprenons le même fichier, mais en spécifiant le troisième paramètre :

```
<?php $fic=fopen("fichier.txt", "w"); //ouverture du fichier en mode écriture, écrasement du contenu s'il existe
fwrite($fic,"Un texte dans votre fichier",8);// insert les 8 premiers caractères de la chaîne : Un texte
?>
```

La fonction fwrite n'est qu'un exemple de fonction pour l'écriture dans un fichier texte mais il y en a encore d'autres.
La fonction fputs() par exemple, ne présente pas de différence importante avec fwrite() c'est juste son alias.
Une autre fonction très intéressante à connaitre est la fonction file_put_contents. Elle prend en paramètre une adresse de fichier et une chaine de caractères. En effet, dans les versions récente de PHP, on a essayé de rendre plus rapide certaines opérations. **Avec la fonction file_put_contents il n'est pas nécessaire d'ouvrir un fichier**. Ce dernier se créera automatiquement ou s'il existe déjà, son contenu sera écrasé.
Fermeture d'un fichier : fclose
Après tout traitement effectué sur un fichier, on doit le fermer par la fonction fclose() comme suit :

```
<php
fclose($fic);
?>
```

Cette fonction retourne true si la fermeture du fichier a réussi et false dans le cas contraire.

Lecture rapide et intégrale d'un fichier avec PHP

La partie précédente nous a permis d'aborder la notion de gestion de fichier en PHP. On a appris à ouvrir et à fermer un fichier, et surtout à écrire dans le fichier. Maintenant, faisons l'inverse. Imaginons un fichier, et que l'on veuille récupérer son contenu pour le traiter ensuite. PHP propose encore une multitude de fonctions pour gérer tout ça.
Plusieurs fonctions permettent de lire tout un fichier en une passe, parmi lesquelles on distingue la fonction file_get_contents().
Son utilisation est assez simple puisqu'il suffit de lui passer en paramètre l'adresse du fichier pour qu'elle retourne une chaîne de caractère avec l'intégralité du contenu.

```
<?php
//lecture intégrale d'un fichier
$contenu=file_get_contents("fichier.txt");
```

```
echo $contenu;
?>
```

Une autre fonction s'appelle readfile(). Elle est identique à la fonction file_get_contents() sauf qu'elle retourne le contenu du fichier vers la sortie standard et retourne le nombre d'octets lus. **On utilise donc la fonction readfile() dans le cas où l'on ne veut pas avoir le fichier dans une chaîne**.

```
<?php
//lecture intégrale d'un fichier
if(readfile("fichier.txt"))// si le fichier a été lu correctement, affichage du contenu sur le navigateur
{
echo "
nombre d'octect :".readfile("monfichier.txt"); //affichage du nombre d'octets
}
else // sinon
{
echo "aucun affichage";
}
fclose($fic) ;
?>
```

Il existe de plus une fonction similaire à readfile() mais qui nécessite l'ouverture de fichier avec fopen. Il s'agit de la fonction fpassthru(). Elle prend comme unique argument le descripteur de fichier retourné par fopen() à l'ouverture.
La fonction file() offre une troisième possibilité afin de lire le fichier en intégrale. Cette fonction est aussi identique à readfile, sauf qu'au lieu de renvoyer le contenu du fichier vers la sortie standard, elle le transforme en un tableau. La syntaxe pour récupérer le contenu est donc celle-ci :

```
$tableau=file($fic);
```

Lire un fichier caractère par caractère : fgetc

La lecture d'un fichier peut également consister à prendre son contenu caractère par caractère, au moyen de la fonction fgetc().

```
<?php
//lecture d'un fichier caractère par caractère
$fic=fopen("fichier.txt", "r");
while(!feof($fic))
{
$caractere=fgetc($fic);
if(!feof($fic))
{
echo $caractere . "<br />";
}
}
fclose($fic) ;
?>
```

Ce code lit un caractère à la fois dans un fichier et l'affiche. Vous remarquerez une nouvelle fonction dans la boucle. Il s'agit de feof() dont le terme est l'abréviation de " File End Of File " (traduite par "fichier : fin de fichier"). **Cette fonction prend en paramètre un pointeur de fichier et retourne true si le pointeur est positionné à la fin du fichier**.
La boucle continue donc tant que ce n'est pas la fin du fichier. Pour que ça soit plus visible, on a mis chaque caractère sur une ligne lors de l'affichage.

Lire un fichier ligne par ligne : fgets

Il est aussi intéressant de récupérer chaque ligne du fichier. La fonction employée est fgets(). Elle retourne tous les caractères jusqu'à la prochaine fin de ligne. Cependant, une telle lecture peut s'avérer lourde si l'on manipule des chaine de caractère de grande taille.
On peut donc spécifier une taille en octets comme deuxième paramètre. **Ainsi, la fonction retourne au maximum ce nombre de caractère même si aucune fin de ligne n'a été détectée**. Habituellement, on spécifie une taille arbitraire de 1024 octets pour lire toute la ligne.

```
<?php
//lecture d'un fichier ligne par ligne
$fic=fopen("fichier.txt", "r");
$i=1 ;//Compteur de ligne
while(!feof($fic))
```

```
{
$ligne= fgets($fic,1024);
echo "ligne numéro ".$i." : " . $line . "<br />";
$i ++;
}
fclose($fic) ;
?>
```

Exploitation d'un fichier de tableur CSV

On n'a pas cessé d'apprendre des choses nouvelles sur la gestion des fichiers en PHP mais à quoi sert tout ça ?
Une des applications très courante de la manipulation de fichier est le traitement de fichier CSV. **En effet, le format CSV ou Comma Separated Values constitue un vrai standard d'échange**.
On a par exemple un fichier CSV qui contient pas mal de données, et on souhaite accéder au contenu pour les manipuler comme des variables PHP.
La fonction fgetcsv() permet d'avoir une ligne d'un fichier CSV. Si vous n'avez pas encore un fichierCSV prêt à emploi, ouvrez votre tableur (Excel par exemple), et entrer les données sur chaque ligne. Ensuite, enregistrez sous format CSV.
Soit un fichier acteur.csv de cette forme :

```
Jakie;Chan;90
David;Caradine;34
Luciela ;Santos;89
Véronica ;Castro;67
```

On utilise la fonction fgetcsv comme suit :

```
<?php
$ligne = 1; // compteur de ligne
$fic = fopen("acteurs.csv", "a+");
while($tab=fgetcsv($fic,1024,';'))
{
$champs = count($tab);//nombre de champ dans la ligne en question
echo "<b> Les " . $champs . " champs de la ligne " . $ligne . " sont :</b><br />";
$ligne ++;
//affichage de chaque champ de la ligne en question
for($i=0; $i<$champs; $i ++)
{
echo $tab[$i] . "<br />";
}
}
?>
```

Compléments : autres fonctions pour la manipulation d'un fichier

A part tout ce que l'on a déjà vu ici, il y a encore d'autres fonctions utiles qui méritent d'être abordées dans ce tutoriel. Bien entendu on n'aura pas le temps de tout voir, mais on a essayé et on essayera d'aborder l'essentiel.

Vérification de l'existence d'un fichier : file_exists

La fonction file_exists() est intéressante puisqu'elle permet de déterminer si un fichier existe ou pas.

```
<?php
if (file_exists("fichier.txt"))
{
echo "Le fichier existe'";
}
else
{
echo "Fichier inexistant";
}
?>
```

Taille d'un fichier : filesize

On peut savoir la taille d'un fichier grâce à la fonction filesize(). La taille retournée est mesurée en octets.

```
<?php
if (file_exists("fichier.txt"))
{
```

```
echo "La taille du fichier est de :" . filesize("fichier.txt")." octets";
}
else
{
echo "Fichier inexistant";
}
?>
```

Suppression d'un fichier : unlink
Puisque c'est la fin des manipulations, profitons pour faire un peu le ménage :
La destruction des contenus d'un fichier est parfois nécessaire, or la fonction unlink() permet de supprimer un fichier. Si le fichier ne peut pas être supprimé, par absence d'autorisation par exemple, la fonction retourne false.

```
<?php
unlink("fichier.txt");
?>
```

En faisant cela, vous remarquerez que le fichier.txt qu'on a utilisé tout au long du tutoriel n'apparait plus dans votre répertoire.
Attention à ne pas supprimer un fichier important, la fonction est irrémédiable ! En considérant tout ça comme étant acquis, passons maintenant à autre chose.

Les bases de données

Après avoir appris à récupérer les données saisies par l'utilisateur, on a cherché un moyen de les stocker dans le but de s'en servir plus tard. Certes, on a trouvé le moyen de le stocker : les écrire dans des fichiers. **Toutefois les stocker dans les bases de données nous permet de faire tout un tas de manipulations que l'on ne pourra que difficilement faire avec les fichiers**.
Déjà, dans une base de données, les données sont plus organisées et structurées de façon ordonnée. Ce qui les rend facilement accessibles. Mais qu'est ce donc une base de données ?
Une base de données est un tout simplement un ensemble de données stockées dans des mémoires de masse sous une forme structurée. Le but de l'établissement d'une base de données est qu'elle soit accessible par des applications différentes et des utilisateurs différents.
Une base de données est d'abord faite pour être gérée. **Le chef d'orchestre pour la gestion d'une base de données est un outil appelé SGBD ou Système de Gestion de Base de Données**. Il est composé d'une base de données et d'un réseau de poste de travail consultant ou mettant à jour les données.
La base de données sera alors composée de plusieurs tableaux appelé tables. Chaque libellé du tableau est appelé champ. Et chaque ligne d'une table est un enregistrement. Pour identifier chaque enregistrement d'une table, il y a ce qu'on appelle clé primaire. C'est en générale une sorte de numéro unique qui différencie un enregistrement des autres enregistrements de la table.
Voici un exemple de table :

Table client

idclient	nomclient	prenomclient	mailclient
1	Brown	James	james@brown.fr
2	Schröder	Helmut	helmut@yes.com

SGBD

Généralement, on parle plutôt de Système de Gestion de Base de Données Relationnel ou SGBDRqui est SGBD permettant d'introduire le concept de relation entre les différentes tables.
Les principaux buts d'un SGBDR sont donc :
Consultation optimisées des données
On peut avoir différentes options classiques sur la manipulation des données : consultation, insertion, modification et suppression.
Gestion des accès aux données
Des systèmes de privilèges sont intégrés dans un SGBD, ce qui leur permettent de gérer l'autorisation d'accès aux données. Un SGBD possède des mécanismes intégrés permettant de gérer les accès simultanés sans que les utilisateurs n'aient besoin de s'en occuper.
Le maintient de la cohérence des données entre elles et le contrôle d'intégrité

La cohérence des données est basée sur le respect des contraintes d'intégrité qui sont des conditions auxquelles doivent satisfaire les données pour être acceptées dans la base. Les contraintes d'intégrité sont contrôlées par le moteur du SGBD :
Au niveau de chaque champ par exemple, l'âge ne doit pas prendre une valeur négative.
Au niveau de chaque table, deux personnes ne peuvent pas avoir à la fois le même nom et le même prénom
Toutefois, la redondance des données n'est pas forcément entièrement perçue automatiquement par les SGBDR. Il faut des manipulations spécifiques pour trouver les anomalies. Ce qui semble être une grosse lacune parce que les erreurs sont déjà présentes dans la base.

Le langage SQL

Par définition, une base de données est structurée mais cette structuration doit avoir un caractère universel. **Autrement dit, il ne faut pas que la structure soit adaptée seulement à une application particulière mais qu'elle puisse être utilisable par des applications distinctes**.
D'où la conception d'un langage standard pour la manipulation des données. Il s'agit du langageSQL ou Standard Query Language. C'est un langage standard de requêtes qui permet d'interroger les bases de données de manière simple. Le SQL est facilement compréhensible par l'humain et par les différents SGBDR.
D'une manière générale, les instructions pour l'interrogation de la base constituent ce que l'on appelle une requête SQL. Une requête SQL pourrait être alors une modification, une insertion, une suppression, ou une sélection de données, voire même une création de table. On verra tout cela de plus près dans la partie suivante.
N'hésitez pas à consulter en détail le **tutoriel SQL** avant de commencer à utiliser une base de données avec PHP.

Système de base de données MySQL

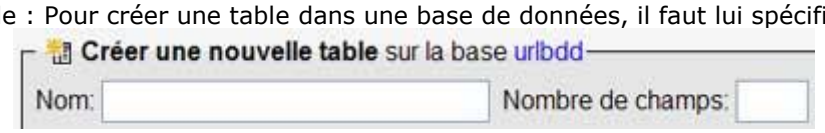
MySQL est un SGBDR qui enregistre des informations par définition même d'une base de données et PHP constitue le langage intermédiaire entre cette base et l'utilisateur de la base en question. **Dans ce chapitre nous verrons dans le cas pratique la manipulation d'une base de données**. Pour cela nous allons l'illustrer à partir de divers exemples. Ces derniers seront traités sur phpmyadmin d'**easyPHP**.

Création d'une base de données

La première étape de notre traitement sera donc de créer la base de données elle-même. Avant tout n'oubliez pas de voire si easyPHP est bien démarré. Sinon, il faut le faire. **Une fois qu'easyPHP est en marche, allez dans l'espace administration et cliquer sur "gestion de base de données"** pour aller vers l'interface phpmyadmin.
Dans l'interface phpMyadmin, créer une base de données en spécifiant son nom : base1



Création de table : Pour créer une table dans une base de données, il faut lui spécifier son nom.



Spécification des champs



Chaque table d'une base de données comporte des informations relatives à un même sujet et chaque colonne ou champ d'une table contient des faits individuels relatifs au sujet de la table.
En esquissant les champs des tables, il faut garder à l'esprit qu'il doit y avoir une relation directe entre chaque champ et les sujets de la table. **Il est fortement recommandé de ne pas inclure des données dérivées ou calculées**. C'est à dire des données qui sont les résultats d'une expression. On a toujours intérêt à stocker les informations de la manière la plus basique et la plus logique possible.
Les champs ont chacun un nom unique et ils sont tous associés à un type de données particulier. Les champs peuvent donc être de type texte, entier, booléen, etc selon la nature des données qu'ils vont contenir.
Quelques exemples de types de données en Mysql

- **VARCHAR** : Chaine de caractères
- **BOOL** : de type booléen (vrai / faux)
- **DATE** : de type date
- etc.

Dans une table élève par exemple, voici un exemple de ce que pourrait être une liste de champs :

Table élève				
Champs	**type**	**taille/valeur**	**Signification du type**	**Signification du champ**
numero	INT	5	entier	Numéro d'identification de l'élève
nom	VARCHAR	30	chaine de caractère	Nom de l'élève
prenom	VARCHAR	30	chaine de caractère	Prénom de l'élève
adresse	TEXT		texte	Adresse de l'élève

Remarquons que certain type peut avoir une taille / valeur. C'est le cas du type VARCHAR par exemple. Spécifier un type VARCHAR de taille 30 pour le champs nom signifie que le nombre de caractère pour un nom ne peut pas dépasser 30.

Clé primaire d'une base de données

Après avoir créé les champs, il nous faut définir un moyen d'identifier chaque élève. Cette valeur doit être unique. Si on prend les champs nom par exemple, des élèves issu de la même famille peuvent avoir le même nom, donc impossible de les identifier juste par leur nom. On prendra alors le numéro de l'élève comme clé primaire de la table élève. Pour cela, passer la table en mode structure et cliquer sur primary ou sur la petite clé la version de votre logiciel.

La requête de création de table

Comme l'on a déjà dit dans le chapitre précédent, les requêtes SQL sont des moyens d'interrogation de la base de données. Concrètement, **c'est via les requêtes SQL que l'utilisateur passe ses commandes à la base de données** en lui disant ce qu'il veut avoir : Une requête traduite en langage humain serait quelque chose du genre "*Donnez-moi s'il vous plait la liste de tous les élèves qui se trouvent dans la table eleve*".
Ainsi le SGBDR est là pour nous servir. On peut lui demander de nous remettre une liste de données, d'insérer une liste de données, etc.
Précédemment, on a créé la table manuellement via l'interface phpMyadmin, mais il y a aussi un autre moyen pour créer une table. Nous pouvons nous même dire directement au système de créer la table à l'aide d'une requête SQL.

```
CREATE TABLE `ecole`.`eleve` (
`numero` INT( 5 ) NOT NULL ,
`nom` VARCHAR( 30 ) NOT NULL ,
`prenom` VARCHAR( 30 ) NOT NULL ,
`datenaissance` DATE NOT NULL ,
`adresse` TINYINT NOT NULL ,
PRIMARY KEY ( `numero` )
)
```

Dans cet exemple ecole est la base de données et eleve est le nom de la table. **Le mot clé à retenir pour la création de table est CREATE TABLE**. Il ne faut pas oublier de séparer la définition de chaque champ par une virgule.

Insertion de données dans un table : INSERT INTO

En spécifiant le nom de la table, la commande INSERT INTO permet d'insérer des données sous forme d'enregistrements dans une table. Les valeurs à insérer sont listées dans la clause values et séparées par des virgules.

```
INSERT INTO `eleve` (
`numero` ,
`nom` ,
`prenom` ,
`adresse`
)
```

```
VALUES (
'1', 'Cassidy', 'Hope', 'Appartement 4 21000 Rue de la charité Pays'
);
```

Sélection de données : SELECT

On peut avoir la liste de tous les élèves de la table élève en utilisant le mot clé SELECT, suivie de la clause FROM, puis du nom de la table.
La requête associée est donc :

```
SELECT * FROM `eleve` ;
```

L'utilisation du signe * veut dire que l'on veut avoir tout les enregistrements. Dans le cas où on veut seulement avoir les enregistrements répondant à certains critères, on emploie le mot clé WHERE.
On spécifie une certaine condition dans la clause where :

```
SELECT * FROM `eleve` WHERE `numero` =1.
```

Cette requête demande à la base de données de sélectionner l'élève numéro 1, et de nous retourner le résultat.
Il peut y avoir plusieurs conditions :

```
SELECT *
FROM `eleve`
WHERE `nom` LIKE 'Cassidy'
AND `prenom` LIKE 'Hope'
```

Traduite en langage humain, cela signifie qu'il faut sélectionner et afficher tout les élèves dont le nom est "Cassidy" et dont le prénom est "Hope"

Modification de données : UPDATE

Le mot clé utilisé pour la modification des données est UPDATE. **En l'associant à la clause WHERE, il est possible de rechercher des données ayant certains critères et les modifier**.
Pour modifier le prénom de l'élève numéro 1 par exemple on utilisera la syntaxe suivante :

```
UPDATE `eleve` SET `prenom` = 'Vanessa' WHERE `eleve`.`numero` =1 ;
```

Il est aussi possible de modifier plusieurs champs d'un seul coup avec cette syntaxe :

```
UPDATE `eleve` SET `nom` = 'Paradis', `prenom` = 'Vanessa' WHERE `eleve`.`numero` =1 ;
```

Ici, on modifiera le nom et le prénom à la fois. Il ne faut pas oublier que les champs sont toujours séparés par des virgules, contrairement aux conditions, qui sont séparées par des opérateurs booléens (AND, OR, ...)

Suppression de données : DELETE

On peut procéder de la même façon pour supprimer un (ou plusieurs) enregistrement(s) sauf que cette fois ci on utilisera les mots clé DELETE FROM.

```
DELETE FROM `eleve` WHERE `numero` =1 ;
```

La requête signifie qu'il faut supprimer de la table eleve l'élève dont 'numéro' = '1'.

Insertion des données saisies

Après avoir lu les chapitres sur la base de données Mysql dont l'intérêt n'est pas négligeable, revenons à nos moutons. Appliquons maintenant notre savoir faire, en essayant de mettre en évidence l'interaction entre PHP et Mysql qui est le but même de cette partie.
Dans un premier temps, nous allons suivre pas à pas les étapes de la récupération des données d'un formulaire pour l'introduire ensuite dans la base.
Dans un second temps, nous allons faire l'inverse. Nous sélectionnerons des données dans la base que l'on va ensuite récupérer à l'aide de PHP et ensuite on l'affichera dans un tableau html.
Pour y voir plus clair, nous allons séparer dans un premier temps le formulaire HTML du traitementPHP. Afin d'éviter de trainer sur des sujets qui dépassent le cadre de ce chapitre, on ne va pas faire une explication détaillé sur les formulaires.
Puisque cette notion a déjà été abordée dans un des chapitres précédents, on se contentera d'utiliser un formulaire tout fait. Il est conseillé à ceux qui en ont besoin de se rendre dans les**tutoriels XHTML** pour en savoir plus sur les **formulaires**.
Code XHTML du formulaire dans formulaire.html

```
<form method="post" action="traitement.php">

   <legend>Renseignement sur un élève</legend>

   <label for="nom">Nom:</label>
```

```
    <input type="text" name="nom" id="nom" />

    <label for="prenom">Prénom:</label>
    <input type="text" name="prenom" id="prenom" />

    <label for="adresse">Adresse:</label>
     <textarea name="adresse"></textarea>

<input type="submit" value="envoyer" />
</form>
```

Voici donc un formulaire de saisie qui nous permettra de saisir des élèves.

Traitement des données saisies

Une fois que l'on aura saisi les données, l'étape suivante consistera à les récupérer et à les insérer dans la base de données.

Connexion à la base

Avant tout, il est quand même important de savoir qu'à chaque fois que l'on veut effectuer un traitement mysql en PHP, il nous faut ouvrir et fermer une connexion à la base. Que se soit une insertion de données dans la base ou une sélection, toute communication avec la base nécessite la connexion à la base de donnée tout au début de la manipulation.

Une fois que l'on en aura fini avec la base, il faudra libérer et fermer cette connexion.

Les codes de connexion et de déconnexion risquent d'être utilisé partout où il y a une communication avec la base. C'est pourquoi on a intérêt à les mettre sous forme de fonction, et à faire en sorte que les fonctions soient accessibles partout. Et voilà le bon retour des termes de fonctions et inclusion de fichier dont on ne peut jamais se passer.

De nombreuses fonctions de base sont utilisées pour la connexion, en voici quelques une qui sont bonnes à savoir :

- Connexion au serveur: mysql_connect()
- Sélection de la base de données : mysql_select_db()
- Fermeture de la base : mysql_close()
- Code dans fonctionBD.php

```
<?php
//ouverture de la connexion
function connect_bd()
{
   $nomserveur='localhost'; //nom du seveur
   $nombd='ecole'; //nom de la base de données
   $login='root'; //login de l'utilisateur
   $pass=''; // mot de pass
   $bd=mysql_connect($nomserveur, $login, $pass)or die("Connexion échouée");
   mysql_select_db($nombd,$bd)or die("La base ne peut pas être selectionnée");
   return $bd;
}

//fermeture de la connexion
function deconnect_bd($bd)
{
   mysql_close($bd);
   $db=0;
}
?>
```

Ces deux fonctions connect_bd et deconnect_bd servent respectivement à l'ouverture et à la fermeture de la connexion. Le fichier sera alors inclus à chaque fois que l'on aura besoin d'une communication avec la base.

Traitement de données

Passons maintenant au vif du sujet. On a maintenant les données, il ne nous reste plus qu'à les récupérer, communiquer avec la base et les insérer dans la base. Tout cela va se dérouler dans un fichier traitement.php.

```
<?php
//inclure les fonctions pour la base
include("fonctionBD.php");
```

```
//récupération des données et les formater un peu
$nom = addslashes($_POST["nom"]);
$prenom = addslashes($_POST["prenom"]);
$adresse = addslashes($_POST["adresse"]);

//Insertion de données
$bd=connect_bd();
if($bd<>0)
{

   $requete="insert into eleve VALUES ('', '".$nom."','".$prenom."','".$adresse."')";
   $resultat=@mysql_query($requete,$bd);
   if($resultat<>FALSE)
   {
      echo 'Enregistrement réussi, merci. Les valeurs insérées sont :<br />';
      echo $nom.'<br />';
      echo $prenom.'<br />';
      echo $adresse.'<br />';

   }
   else
   {
      echo "Echec de l'enregistrement, désolé";
   }
   deconnect_bd($bd);
}
?>
```

Après l'inclusion des fichiers, on a récupéré les données à l'aide du tableau $_POST qui n'est plus un inconnu pour nous.
Il est vrai qu'il y a encore un certain nombre de choses que l'on a zappé, comme les vérifications des champs saisi par exemple. Entre temps, le javascipt peut s'occuper de cela. **Néanmoins, on a essayé d'être un petit peu prudent et on a utilisé la fonction addslashes**. Cette fonction rajoutera des antislashs dans une chaîne là ou il y a des caractères à échapper. Si jamais l'utilisateur nous a par exemple rentré une adresse telle que " avenue de l'indépendance ", la fonction nous rajoutera automatiquement un antislash avant le guillemet afin d'éviter toute erreur. Ceci n'est qu'un exemple, mais de bon nombre de formatages peuvent et doivent être effectués avant que les données ne soient prêtes à être insérées dans la base : **Laisser un internaute remplir des champs que l'on insère dans la base de données présente un risque potentiel, tant dans l'intégrité des données que du point de vue de la sécurité**.
Ensuite vient le tour de l'insertion de la base. C'est la fonction mysql_query qui s'occupe de l'envoie de la requête d'insertion au serveur.
Remarque : Pour mieux gérer la clé primaire, on a mis le champ numéro en auto incrément. Pour ce faire, passer dans la structure du champ le paramètre 'extra(en auto incrément. On n'aura plus besoin alors de saisir le numéro, il s'incrémentera automatiquement à chaque insertion d'un enregistrement.

Sélection et Affichage des données

Maintenant, faisons l'inverse. Après avoir inséré bon nombre de données dans la base, essayons de les sélectionner et de les afficher.
Code dans selection.php

```
<?php
//select
$bd=connect_bd();
if($bd<>0)
{
   $requete="select * from eleve";
   $resultat=@mysql_query($requete,$bd);
   if($resultat<>FALSE)
   {
      echo "<table border=\"1\">";
      //entête du tableau
      echo "<tr>";
      echo "<th>Numero</th>";
            echo "<th>Nom</th>";
```

```
echo "<th>Prénom</th>";
      echo "<th>adresse</th>";
      echo "</tr>";
   //fin de l'entête

   while ($row=mysql_fetch_array($resultat))
   {
     echo "<tr>";
      echo "<td>".$row['numero']."</td>";
      echo "<td>".$row['nom']."</td>";
      echo "<td>".$row['prenom']."</td>";
      echo "<td>".$row['adresse']."</td>";
     echo '</tr>';
   }
   echo "</table>";
}
else
{
      echo "Echec de l'enregistrement, désolé";
}
deconnect_bd($bd);
}
?>
```

La fontion mysql_fetch_array() nous retourne une ligne de résultats sous la forme d'un tableau, dont les clés sont les noms des champs sélectionnés (numero, nom,…)

Envoi d'email en PHP

PHP est utilisé dans un environnement web. Par conséquent, il doit permettre la communication sur internet, notamment l'envoi de emails.
Il existe différents moyens pour envoyer un email en PHP, le plus simple étant d'utiliser la fonction prévue à cet effet, à savoir la fonction mail().

La fonction mail

La fonction mail() s'utilise de cette façon :

```
<?php
mail('email_destinataire', 'sujet', 'message');
?>
```

mail() retourne TRUE si le mail est envoyé et FALSE dans le cas contraire**.
La fonction mail () envoi automatiquement le texte "message" au destinataire(s).
Si l'on veut envoyer un mail à plusieurs destinataires, il faut séparer les adresses par des virgules.
Exemple de script :

```
<?php
mail(
"personne@domain.com",
"test d'envoie de mail",
"Ceci est un test."
);
?>
```

L'envoi d'emails avec pièces jointes ou contenus particulier (par exemple emails en HTML), pourra être également effectué avec la fonction mail(). Cependant il faut respecter les règles de l'encodage MIME.

Entête MIME des emails

Pour envoyer un mail il est nécessaire de définir un entête MIME. **Les entêtes des messages Internet obéissent à des normes imposées par les RFC** (Requests For Comments) et l'objectif de MIME est de définir un standard au niveau international. On peut alors ajouter des entêtes supplémentaires dans la fonction mail.
Différentes possibilités s'offrent à nous pour ce qui est du contenu à mettre dans l'entête d'un mail.
Voici quelques champs d'entêtes :
Subject : Le sujet du message.
From : L'email de l'auteur du courrier.
Sender : L'adresse de l'expéditeur (couriel@domaine.com)
Reply-To : l'adresse de réponse au mail (reponse@domaine.com)
To : Liste des adresses des destinataires du mail (destinataire1@domaine.com, destinataire2@domaine.com)

Cc : La liste des destinataires que l'on met en copie du mail(copie@domaine.com)
Bcc : Les destinataires à qui on envoie une copie du mail, mais que l'on ne voit pas apparaitre dans la liste des emails en copie (invisible@domaine.com)
Message-ID : Code unique d'identification du message (code)
In-Reply-To : utilisé pour identifier le (ou les) mails dans le cas des emails de réponse. (message_id)
References : utilisé pour identifier le fil de la conversation. (message_id)
Comments : Commentaires à propos du mail.(commentaires)
X-Mailer : Le logiciel qui a envoyé le mail (PHP).
Keywords : Mot-clé mots-clés relatifs au mail.
Date : Date d'expédition (date)
MIME-Version : Version MIME du mail (1.0)
Content-Type : type/sous-type du contenu
Content-transfer-encoding : Encodage du contenu d'un mail.
Content-ID : message_id la référence du contenu d'un autre mail.
Content-Description : texte. Information descriptive à propos du contenu d'un mail.
Exemple d'envoi d'email avec entêtes :

```
<?php
//envoie de mail avec entête
$expediteur = "admin@domain.com";
date("D, j M Y H:i:s"); //date
$entete = "From: $expediteur\n"; // expéditeur
$entete .= "Cc: \n";
$entete .= "Reply-To: $expediteur \n"; // Adresse de retour, retour à l'envoyeur en cas d'erreur
$entete .= "X-Mailer: PHP/" . phpversion() . "\n" ; //version
$entete .= "Date: ". date("D, j M Y H:i:s"); //date;
mail( "tout@domain.com", "Communication", "Nous vous communiquons que le département sera fermé tous les mardi après midi.", $entete);
>>
```

La fonction email

Parfois on utilise la fonction email() en remplacement de la fonction mail(). Cette fonction, lorqu'elle existe, permet uniquement d'envoyer des mails à partir de votre adresse email.
Son utilisation est à peu près similaire à celui de la fonction mail() mais il y a quelque peu de différence au niveau de la syntaxe :
email(chaîne compte, chaîne destinataire, chaîne titre, chaîne message [,options]);
** ATTENTION :
Une confusion est souvent faite entre le fait que la fonction retourne 'true', et le mail ne part pas. En fait, la fonction 'mail' retourne 'true' si le parser PHP réussit bien à envoyer le mail. Ceci ne veut pas dire que le mail part. Il existe beaucoup d'autres facteurs influant sur l'envoi de mails.
Si 'php' arrive à envoyer le mail, alors ce mail part dans le serveur (généralement 'apache'), et la fonction 'mail' renvoie 'true'.
Après, le problème de savoir si 'apache' arrive à envoyer le mail ou pas n'est plus le problème de php, mais de 'apache'.
Donc il est vain de tester que 'mail' retourne 'true', dans la mesure où... il retourne toujours 'true'... A moins que le parser php ne soit pas correctement installé.

Créer un formulaire et le traiter avec PHP

Encore une fois on retrouve la notion de formulaire qui est un des points les plus important dans la réalisation d'un site web dynamique. C'est un moyen très adaptés pour réaliser une attractivité entre le visiteur et le site en question.
De la saisie jusqu'à la récupération des données, nous allons voir toutes les notions de base qu'il faut connaitre lorsque l'on manipule les formulaires. Dans ce contexte, on verra une utilisation fréquente des super globales.
Etant donnée que ce sujet a déjà été traité dans les **tutoriels XHTML** et **CSS**, on ne va pas s'y attarder. Rappelons juste qu'il faut utiliser la balise form pour créer un formulaire. A l'inverse, nous allons insister un peu sur les choix des méthodes d'envoi des formulaires.
En effet, les deux valeurs possibles de l'attribut méthode nous permettent d'avoir deux méthodes possibles pour envoyer les données dans un formulaire : soit par la méthode GET soit par la méthode POST.
Les deux méthodes sont à peu près les mêmes puisque toutes les deux permettent d'envoyer des données. Toutefois, chacun a sa façon de gérer les choses et le choix des méthodes utilisées nécessite la vigilance du développeur.

Formulaire avec méthode GET

La méthode GET consiste à transmettre les données via l'URL. Il est intéressant de l'utiliser dans le cas où on veut faire transiter des données de pages en pages. Cependant, il faut avoir conscience que les données seront transmises par http et donc visibles dans la barre d'adresse.
Cet aspect mérite une attention particulière parce que de la sécurité des données est menacée. Dans le cas où les données sont confidentielles, mieux vaut chercher à ne pas utiliser cette méthode. Si toutefois on ne peut s'en passer, il faut essayer de cacher les valeurs transmises en utilisant les cryptages par exemple.

Formulaire avec méthode POST

Une autre méthode pour l'envoi des formulaires est la méthode POST. Il est obligatoire de l'utiliser dans certains cas comme l'envoi de fichiers, envoi de données lourdes, envoi de données confidentielles. **La méthode POST est également recommandée si le formulaire déclenche une action spécifique qui doit être renouvelée à chaque fois**. Avec la méthode POST, il est possible de revenir en arrière et de re-soumettre le formulaire.

Récupération des données

Dans l'attribut action d'un formulaire, on spécifie l'adresse du fichier dans lequel les données sont renvoyées et traitées.
Utilisation des super globales
Les données transmises se trouvent souvent sous forme de tableau dans des variables qu'on appelle des **superglobales** :
Le tableau $_GET permet de récupérer toutes les données envoyées par la méthode GET. Il contient toutes les données transmises via l'URL.

- Le tableau $_POST contient les données envoyées par la méthode POST.
- Le tableau $_FILES est spécifique à l'envoi de fichiers. Il contient les informations du fichier quand l'utilisateur télécharge un fichier via un formulaire.
- Comme pour tout tableau, l'affichage des contenus de ces tableaux se fait par la fonction print_r().
- Cela pourra nous donner un aperçu des données qui sont transmise.

En faisant :

```
<?php
echo '<pre>' ;
print_r($_POST) ;
echo '</pre>' ;
?>
```

On a la liste de toutes les valeurs transmises via la méthode POST. Mais ce qui nous intéresse le plus, c'est d'avoir accès à ces données une à une. C'est ce que nous allons voir par la suite.
Exemple de formulaire utilisant la méthode POST :
Code dans formulaire.html

```
<form method="post" action="questionnaire.php">
<fieldset>
<legend>Coordonnées</legend>
<label for="titre">Titre : </label>
<select name="titre">
<option value="Mme" selected="selected">Madame</option>
<option value="Mlle">Mademoiselle</option>
<option value="Mr">Monsieur</option>
</select>

<label for="nom">Nom:</label>
<input type="text" name="nom" id="nom" />

<label for="prenom">Prénom:</label>
<input type="text" name="prenom" id="prenom" />

<label for="email">Email:</label>
<input type="text" name="email" id="email" />

<label for="tel">Téléphone:</label>
<input type="text" name="tel" id="tel" />

</fieldset>
```

```
<fieldset>
<legend>Centres d'intérêt</legend>
<p>

Quels sont vos centres d'intérêt ?&tt;br />

<input type="checkbox" name="musique" id="musique" /> <label for="musique">musique</label>
<input type="checkbox" name="politique" id="politique" /> <label for="politique">politique</label>
<input type="checkbox" name="informatique" id="informatique" /> <label
for="informatique">informatique</label>
<input type="checkbox" name="cuisine" id="cuisine" /> <label for="cuisine">cuisine</label> </p>
<</fieldset> input type="submit" value="envoyer" /> </form>
```

Code dans questionnaire.php

```
<?php
//récupération des données simple
echo $_POST['titre'].'
';
echo $_POST['nom'].'
';
//récupération des cases à coché
if(isset($_POST['musique']))// si la case musique est cochée, c'est que le formulaire a envoyé quelque
chose
{
  echo $_POST['musique'];
}
else
{
  echo "Musique non coché";
}
?égt;
```

Pour récupérer une donnée, on utilise généralement le nom du champ comme index du tableau $_POST. La fonction isset() permet de savoir si une variable a été affectée ou non. Si vous ne cochez pas la case musique, la variable $_POST['musique'] ne contient pas de valeur. **Et puisqu'on en parle, profitons pour voir ce qu'est la fonction empty().**
La fonction 'empty()' a le même rôle que la fonction isset() sauf qu'elle renvoie true lorsque la variable vaut 0 ou non définie.

Téléchargement d'un fichier

Pour télécharger un fichier, le champ XHTML associé est un input de type FILE.
Voici un exemple de code qu'on mettra dans un fichier upload.html

```
<form action="enregistre.php" method="post" enctype="multipart/form-data">
<p>
<input type="file" name="fichier" size="40" />
<input type="hidden" name="MAX_FILE_SIZE" value="2048">
<input type="submit" value="Envoyer" />
</p>
</form>
```

Une fois le formulaire validé, les valeurs seront postées dans enregistre.php donc voici le code :

```
<?php $nom_fichier= $_FILES['fichier']['tmp_name'];
echo "nom temporaire:".$nom_fichier;
$nom_destination='C:\wamp\www\img\image.jpg';
move_uploaded_file($nom_fichier, $nom_destination);
?>
```

La variable globale utilisée pour les fichiers est $_FILE .
Au moment du téléchargement, PHP essaie de mettre le fichier dans un répertoire temporaire et lui donne un nom temporaire. C'est la fontion move_uploaded_file qui permet de déplacer le fichier dans le répertoire dans lequel vous voulez le mettre. Ensuite, vous pouvez lui spécifier un nom à votre aise.
Le champ caché 'MAX_FILE_SIZE', quant-à lui, permet de spécifier la taille maximum du fichier que l'on va télécharger.

Les cookies en PHP

Le but de la conception des cookies était au départ de pouvoir ajouter à la fonctionnalité du protocole HTTP la possibilité d'établir un lien entre les différentes requêtes. **Les cookies nous**

offrent la possibilité de retenir des informations sur un visiteur. On pourra ainsi enregistrer des données associées à un utilisateur particulier.
Grâce aux cookies, on peut par exemple retenir le login et le mot de passe d'un utilisateur pour lui éviter d'avoir à le ressaisir lors de sa prochaine connexion. **A l'aide des cookies, on pourra aussi identifier chaque utilisateur de façon unique lors de ses visites**. Ces informations pourront être utilisées à des fins statistiques (traçabilité).

Envoi d'un cookie avec PHP

Lorsque vous demandez une page web, le serveur envoie une valeur avec la page qu'il vous demandera de renvoyer lors de vos prochains échanges. Il est toutefois possible de ne pas les accepter. La plupart des navigateurs, ont des options pour refuser les cookies.
Les cookies sont en réalité des fichiers texte stockés par le navigateur sur l'ordinateur de l'utilisateur, à la demande du serveur.
Si vous ne fixez pas une durée d'expiration pour les cookies, les informations seront stockées dans la mémoire vive de l'ordinateur. Dans le cas contraire, un fichier sera créé sur le disque dur du visiteur. Il s'agit d'un simple fichier texte qui est lisible à partir d'un simple bloc note.
La fonction PHP setcookie() permet de définir un cookie. **On doit appeler cette fonction avant toute balise** HTML ou Header, parce que les cookies doivent passer avant tout autre en-tête. Sinon non seulement le cookie ne sera pas envoyé, mais en plus vous aurez le message d'erreur suivant :
Warning : Cannot send session cookie, header already sent.
Voici un exemple simple d'envoi de cookies

```
<?php
setcookie('moncookie',8)
?>
<html>
<head>
<title>Titre</title>
</head>
<body>
<p>
Vous avez envoyé un cookie
</p>
</body>
<html>
```

Le premier paramètre de la fonction setcookie correspond au nom du cookie, le deuxième à sa valeur. Mais la fonction peut aussi avoir d'autres paramètres comme la date d'expiration, le chemin d'accès, etc :

```
setcookie('moncookie', 5, mktime(11,33,00,04,01,2008),'/cookies');
```

Microsoft Internet Explorer stocke les cookies dans un dossier temporaire de windows(C:\Documents and Settings\votre_login\Local Settings\) .
Voici un exemple de contenu d'un fichier de cookies par exemple :

```
moncookie
5
www.domaine.com/cookies
1024
448989986
31538642
3662392528
29605282
```

Lecture d'un cookie

Si vous avez envoyé un cookie "moncookie" à l'aide de la fonction setcookie à partir d'un site donné, le navigateur renverra le cookie "moncookie" lorsqu'il essaiera d'accéder de nouveau à la page. Le tableau superglobal $_COOKIES permet d'accéder à tous les cookies.

```
<html>
<head>
<title>Titre</title>
</head>
<body>
<?php
if(isset($_COOKIE['moncookie']))
{
```

```
  echo "Le cookie envoyé s'appelle moncookie et sa valeur est : ".$_COOKIE['moncookie'];
}
else
  echo "aucun cookie n'a été trouvé sur cet ordinateur";
?>
</body>
<html>
```

Le tableau $_COOKIE[] est en lecture seule. **Si vous ajoutez un élément au tableau, un cookie ne sera pas envoyé au navigateur**. Seule l'utilisation de la fonction setcookie permet d'en envoyer un.

Validité et date d'expiration des cookies

A défaut d'une précision particulière, la durée de vie d'un cookie se limite à une session de navigateur. Cela signifie que quand vous fermez tous vos navigateurs puis que vous retournez sur la page, le cookie ne sera plus envoyé. **Par contre, il sera toujours possible, à partir du serveur, d'en renvoyer un, et de rétablir le 'dialogue'.**
Pour avoir un cookie permanent, on doit spécifier une date d'expiration sous forme de timestamp.

Limitation des cookies

Limitation aux niveaux du navigateur
Il est convenu qu'il ne faut pas dépasser la limite de 20 cookies par domaine et de 4 ko par cookie. Il est donc impossible d'utiliser les cookies pour stocker de longs textes ou des images.
De plus, les utilisateurs deviennent de plus en plus sensibles aux intrusions. Par conséquent les navigateurs sont maintenant dotés d'un filtre à cookies. **Au cas où vos cookies sont trop volumineux, trop nombreux, ou ayant des dates d'expiration trop loin, il est possible que le navigateur décide de ne pas les stocker**. Si la restriction ne vient pas du navigateur lui-même, c'est l'utilisateur qui le configure de façon à ce qu'il y ait un contrôle total sur les cookies voir même les refuser.
Il ne faut donc pas trop compter sur les cookies pour les informations critiques ou pour des fonctionnalités importantes
Insécurité
Le faite qu'un cookie soit retenu en mémoire et qu'il peut être lu facilement constitue une grande menace pour la sécurité des données. Afin de préserver la sécurité, il est vivement déconseiller de mettre une quelconque information confidentielle dans un cookie.
Il est donc exclu d'y stocker un mot de passe. En effet, quelqu'un de malveillant ayant accès à la machine pourra lire les informations. Ce qui fait que l'utilisation des cookies se limitent aux statistiques à l'assistance à l'utilisateur, ou au stockage des préférences d'affichage du site.

Les sessions et PHP

On entend régulièrement que http est un protocole sans état ou 'statless'. Cela veut dire que le protocole HTTP ne comprend pas de mécanisme de maintien des états entre deux transactions. Lorsqu'un utilisateur demande une page et puis une autre, HTTP n'offre aucun moyen de spécifier que les deux requêtes émanent du même utilisateur.
Le contrôle de session vise à permettre le suivi d'un utilisateur tout au long d'une session sur un site. Les cookies nous ont déjà permis de faire cela mais avec une certaine limitation au niveau des tailles et avec le risque de contrôle par l'utilisateur. Pour la sauvegarde de données confidentielles ou importantes les sessions sont plus adaptées que les cookies.
Leurs applications sont diverses : authentification d'un visiteur, gérer le panier d'achat d'un client sur une boutique en ligne, etc. On peut ainsi envisager la mise en place de formulaires en plusieurs étapes (page par page) donc les données ne seront pas postées d'une page à une autre mais gardées en session, jusqu'à la validation finale. Dans ce chapitre nous allons voir une à une les principales étapes d'un contrôle de session.

Démarrage d'une session

De base, chaque session est caractérisée par un numéro d'identification ID qui permet d'enregistrer des variables de session. Les contenus des variables de session sont stockés sur le serveur.
L'ID de session est la seule information visible du coté du client. Sous forme d'un nombre aléatoire crypté, un ID de session est générer par PHP lui même. Ce numéro peut être stocké sur l'ordinateur comme un cookie ou bien passé en paramètre par l'URL.
Avant d'exploiter la fonctionnalité de PHP relative aux sessions, il faut commencer par ouvrir une session. Trois possibilités nous sont offertes:
La première possibilité est la plus simple de tous. **Elle consiste à appeler la fontion session_start() tout au début du code**. Cette fonction vérifie s'il y a déjà un ID pour la session courante. S'il y en a déjà un, elle charge les variables de session enregistrées pour qu'elles puissent être utilisées dans le corps du programme. Sinon, elle crée un ID de session.

La deuxième possibilité serait d'utiliser la fontion session_register(). Cette option est souvent utilisée dans le cas où la directive register_globals est activée dans le fichier de configuration php.ini.
Une session est automatiquement ouverte lorsque l'on essaie d'enregistrer une variable de session avec cette fonction.
Il nous reste encore une troisième possibilité c'est l'ouverture de session automatique à chaque venue d'un visiteur. Cette option est configurable dans le fichier de configuration php.ini. Cela consiste à définir l'option session.auto_start.

Enregistrement des variables de session

Comme les cookies, les variables de session sont aussi stockées dans un tableau super global. Celui qui correspond aux sessions est $_SESSION. Pour créer une variable de session, on définit un élément dans un de ces tableaux comme suit :

```
$_SESSION['myvar']=5 ;
```

On peut aussi procéder autrement en utilisant la fonction session_register.
Exemple

```
<?php
$var=8 ;
session_register('var') ;
?>
```

Le nom de la variable à enregistrer dont être passée en paramètre de la fonction, et sans le préfixe dollar '$'. La variable var peut alors être utilisée d'un script à un autre jusqu'à la fin de la session.

Utilisation de la variable session

Après avoir ouvert la session, on peut accéder à la variable via le tableau global $_SESSION. Il faut quand même s'assurer que les variables de session ont bien été définies. Vous pouvez par exemple vérifier cela à l'aide de la fonction isset() ou empty().
On peut aussi vérifier que la variable en question est bien une variable de session. La fonction session_is_registered('var') retourne true si la variable qui lui est passée en argument est de type session. Elle retourne false dans le cas contraire.
Toutefois, si on utilise les tableaux super globaux, on n'a pas à utiliser cette fonction. On peut accéder directement au tableau de la manière suivante :

```
if(isset($_SESSION['var']))...
```

Suppression de la session : unset

Une fois qu'on en a terminé avec une variable de session, on peut la supprimer comme suit en cas d'utilisation des tableaux superglobaux :

```
unset($_SESSION['var']);
```

L'idéal étant de vider cette variable auparavant :

```
$_SESSION['var'] = "";
```

Puis

```
unset($_SESSION['var']);
```

Sinon, il est toujours possible de supprimer cette variable à l'aide de la fonction session_unregister();
Après avoir supprimé toute les variables, il ne reste plus qu'à détruire la session complètement en appelant la fonctions session_destroy().

La programmation orientée objet avec PHP

Actuellement, la programmation objet est devenue un grand centre d'intérêt pour les développeurs. Comme les autres langages tel que Java, PHP par son évolution nous offre aussi cette possibilité. Néanmoins, certains d'entre nous ont peut être entendu dire que PHP est largement dépassé par Java sur ce sujet.
Effectivement, dans sa version 4, le modèle objet était très succinct.
Mais soyez rassurés puisque dans sa version actuelle, PHP n'a plus grand chose à envier aux autres langages au niveau programmation objet. La défaillance de la gestion des objets a été largement corrigée lors du passage de PHP 4 à PHP 5. Et tout ça rien que pour nous, alors il serait dommage de ne pas s'en servir.

Pourquoi programmer en objet ?

La programmation en objet n'est pas le but ultime de ce tutoriel, pour autant il s'avère utile d'aborder quelques notions de base pour comprendre la suite.
Commençons par voir en quoi consiste la programmation objet.

Dans la procédure classique, la méthode de programmation utilisée est la programmation structurée. **Cette méthode consiste à décomposer le problème en plusieurs fonctions ou processus**. A l'inverse, la programmation objet décompose le problème sous forme de plusieurs objets ou concept.

Programmation orientée objet	Programmation structurée
On décompose par objet ou concept : - Membre - Livre - Membre	On décompose par Fonction ou Processus : - Enregistrement de prêt - Ajout d'un livre - Gestion des membres

Ainsi, la programmation objet nous donne une vision concrète qui est plus proche du monde réel et nous offre une représentation facile du problème. C'est pourquoi cette méthode nous offre de nombreux avantages dont les principaux sont :

- Code présenté de façon compréhensible : La bonne conceptualisation dans la méthode objet nous permet d'avoir un code naturel, facilement compréhensible et facile à utiliser.
- Réutilisabilité du code : On peut réutiliser un type objet comme étant un modèle pour un autre type en ne réimplémentant que ce qui change entre les deux.
- Modularité du code : La programmation objet nous permet d'isoler facilement un module.

Les Classes

La notion de classe est l'un des grands fondements de la programmation objet. Le principe est de pouvoir construire un modèle de données qui servira de moule pour les objets. Autrement dit, une classe regroupe des objets d'une même famille. **Il existe alors une certaine similarité entre les objets d'une même classe**. Désormais, le terme objet désignera une instance de classe. Imaginons une classe Animal. Les animaux peuvent avoir des comportements similaires. Tout animal dort, mange, boit, évoque des cris, etc. On peut aussi donner des caractéristiques aux animaux (Par exemple : âge, poids, etc)
En termes d'objet, ces caractéristiques sont traduites par ce que l'on appelle les attributs d'une classe. Et on désigne par méthode les comportements des objets. Voyons à l'aide de cet exemple comment créer une classe en PHP.

```
<?php
class Animal
{
public $age;
public $poids;
public $taille;
public $couleur;
function dormir()
{
echo ("Bonne nuit");
}
function manger()
{
echo ("Je mange");
}
function crier($mot)
{
$reponse = "waou ".$mot;
return $reponse;
}
}
?>
```

Pour créer une classe en PHP, on utilise le mot clé class suivi du nom de la classe. PHP ne l'exige pas, mais par respect des normes on mettra toujours les noms de classe avec des majuscules au début.
La classe Animal a donc dans notre exemple 4 attributs : $age, $poids, $taille, $couleur. **Sachant que les attributs d'une classe peuvent être des variables mais aussi des constantes**.
En ce qui concerne la méthode, on les définit à l'intérieur de la classe comme on définit les fonctionsPHP classique. Remarquons aussi qu'une méthode peut retourner une valeur. Nous verrons l'utilisation de ces différents concepts dans le prochain tuto.

Exemple concret de programmation objet sous PHP 5

Comme l'on a vu précédemment, le but même de la programmation orienté est de modéliser de façon naturelle. **Avec une telle méthode, il est possible que le code soit compréhensible facilement par les non informaticiens**.
C'est pourquoi nous avons utilisé l'exemple de l'Animal pour les classes afin de mieux comprendre le concept. Certes, notre but en utilisant la programmation objet n'est pas de pouvoir endormir un animal. Mais de pouvoir manipuler les objets sachant que le principe reste le même et est applicable pour toute sorte d'objet.

Création d'objet PHP

Un compte bancaire peut aussi être considéré comme étant un objet. Créons alors un classe CarteBancaire et essayons d'instancier un objet de cette classe. Pour cela, l'opérateur new joue un rôle important. Il est aussi obligatoire de donner un nom à l'objet.
Voici alors la syntaxe d'instanciation de l'objet $mon_compte avec la définition de la classe correspondante.

```
<?php
//définition de la classe
class CompteBancaire
{
 public $numero ="XXXX";
 public $solde = 0;
}

//instanciation
$mon_compte = new CompteBancaire;
?>
```

Remarques
Ceux qui connaissent Java savent qu'en Java les classes peuvent être définies dans un fichier tout en étant instanciées dans d'autres fichiers du même package sans autre déclaration supplémentaire.
Ceci n'est pas valable en PHP. En PHP, **soit on met la classe dans le même fichier que l'instance, soit on le met dans un autre fichier** et on fait un include ou un require pour inclure le fichier contenant la classe.

Vérification du type d'un objet

Dans le cas inverse, si on n'a que l'objet, comment savoir de quelle classe il tient ? Pour cela, il suffit d'utiliser la méthode get_class() en mettant l'objet en paramètre.
Dans l'exemple précédent :

```
//instanciation
$mon_compte = new CompteBancaire;
echo "Cet objet appartient à la classe: ".get_class($mon_compte) ;
```

Accès à un attribut d'une class

Après avoir créé un objet, voyons maintenant comment l'utiliser. On peut accéder à ces attributs et les manipuler sans problèmes.
Si on veut par exemple afficher la valeur de l'attribut solde de l'objet, on a le code ci-dessous :

```
//instanciation
$mon_compte = new CompteBancaire;
//Affichage solde
echo "Solde: ".$mon_compte->solde;
```

Utilisation d'une méthode d'un objet

De la même façon, on peut aussi faire appel à ces méthodes. Les méthodes peuvent retourner une valeur ou rien.

```
<?php
//définition de la classe
class CompteBancaire
{
  public $numero ="XXXX";
  public $solde = 0;

  function crediter($montant)
  {
   $this->solde += $montant;
  }
```

```
  function debiter($montant)
  {
  $this->solde -= $montant;
  }

  function affiche_solde()
  {
   ;echo "Le solde du compte est : ". $this ->solde;
  }

}

//instanciation
$mon_compte = new CompteBancaire;
//utilisation méthode

$mon_compte->debiter(500); echo "Solde: ".$mon_compte->solde;
?>
```

Une valeur, le principe est le même. Par exemple, on peut ajouter une méthode etat_compte à la classe CompteBancaire, qui retourne l'état actuel du compte.

```
<?php
class CompteBancaire
{
  public $numero ="XXXX";
  public $solde = 0;

  function crediter($montant)
  {
   $this->solde += $montant;
  }

  function debiter($montant)
  {
   $this->solde -= $montant;
  }

  function affiche_solde()
  {
   echo "Le solde du compte est : ". $this ->solde;
 }

  function etat_compte()
  {

   if ($this->solde >0)
   {
     return "créditeur";
   }
   else
   {
     return "débiteur";
  }

 }
}

//instanciation
$mon_compte = new CompteBancaire;

//créditer un compte : le solde vaut après 500
$mon_compte->crediter(500);

//débiter un compte : Le solde vaut après 200
$mon_compte->debiter(300);
$etat = $mon_compte->etat_compte();
```

```
echo "Solde ".$etat." de: ".$mon_compte->solde;
?>
```

La sécurité des fichiers avec PHP

Pendant le traitement (lecture et écriture) de fichier, certains points méritent un peu plus d'attention. Voici quelques astuces et précautions à prendre afin d'éviter les erreurs et garantir la sécurité et l'authenticité des fichiers.

Gérer le tampon : fflush

Quand on écrit des données dans un fichier, elles ne seront pas tout de suite stockées dans le fichier en question. En effet, elles seront d'abord stockées en mémoire avant d'être écrites directement dans le fichier.
Ainsi, si un autre script tente de lire le fichier au même moment, il ne lit pas forcément les données qui viennent d'être ajoutées. Les données ne sont recopiées dedans que par paquets ou à la fermeture du fichier.
La fonction fflush() offre la possibilité d'échapper à cette procédure. Elle est utilisée pour permettre l'écriture de toutes les données bufférisées qui se trouvent dans le tampon mémoire. Autrement dit, la fonction envoie tout le contenu généré dans un fichier, et retourne TRUE en cas de succès, FALSE sinon. Voici un exemple simple d'utilisation :

```
<?php
$fp = fopen($fichier, 'w');
fputs($fp,$donnees);
// Les données sont peut être encore gardées en mémoire
fflush($fp);
// Les données sont certainement écrites
fclose($fp);
?>
```

La gestion du tampon trouve alors son intérêt sur l'assurance de synchronisation.

Gérer les accès concurrents : flock

PHP permet l'accès multiple à un fichier. Ainsi, plusieurs scripts peuvent écrire dans un fichier en même temps. Il en est de même pour la lecture. **Dans ce cas, si deux scripts accèdent à un même fichier de façon simultanée, chacun fait comme bon lui semble.** Il est fort possible qu'il n'y ait plus de cohérences dans le contenu. Le fichier peut alors se trouver incomplet, voire même corrompu.
Une précaution à prendre afin d'éviter ce type de problème serait de verrouiller le fichier pendant son utilisation et de ne relâcher le verrou que lorsque le traitement est fini. **Les scripts accèdent alors un à un au fichier et le fichier n'est plus accessible par les autres scripts pendant son utilisation**.
La fonction flock() permet de mettre ce verrou. Elle prend en argument le descripteur de fichier et un autre argument : le type de verrou (LOCK_SH pour un verrou partagé et LOCK_EX pour un verrou exclusif). Pour relâcher le verrou, il suffit de mettre à la place du type de verrou la valeur LOCK_UN.
Voyons tout cela plus clairement à l'aide d'un exemple.

```
<?php
$fp = fopen($fichier, 'w');
flock($fp,LOCK_SH); // Fichier verrouillé

flock($fp,LOCK_UN); //Fichier déverrouillé
fclose($fp);
?>
```

Sécurité et fichiers

Permission et droit d'accès : chmod

Nous avons vu qu'on peut avoir un système de permission pour les droits d'accès à un fichier. **La fonction chmod() vous permet quant-à elle de modifier facilement les permissions**, en indiquant en argument les valeurs numériques correspondantes.
L'exemple de code ci-dessus rend le fichier "fichier.txt" accessible à la fois en écriture, en lecture, etc.

```
<?php
chmod("/home/user/repertoire/fichier.txt", 0777);
?>
```

Les propriétaires de fichiers et les groupes : chown et chrgrp

Pensez à bien définir qui sont les propriétaires de tels ou tels fichiers et à gérer les groupes auxquels appartiennent les utilisateurs. **Pour une modification, vous avez à votre disposition les fonctions chown() et chrgrp()**. Seulement, vous devriez être un super utilisateur ou root pour pouvoir effectuer ce genre de modification.
De même, seul un propriétaire du fichier et à la fois membre du groupe peut modifier le groupe.

Autres contrôles : safe_mode et open_basedi

Nous attirons maintenant votre attention sur deux directives de configuration, dont l'activation influe beaucoup sur la gestion de sécurité du fichier. Il s'agit de safe_mode et open_basedir.
Pour éviter qu'un utilisateur puisse lire et écrire sur les fichiers des autres, la directive open_basedir effectue un contrôle complémentaire. **Cette directive s'assure que tous les fichiers auxquels on va essayer d'accéder sont bien au bon endroit** (C'est-à-dire dans le sous-répertoire du répertoire spécifié dans la configuration).
Le "Safe Mode" quand à elle est le mode de sécurité de PHP. A chaque fois que l'on accède à un fichier, PHP vérifie si l'utilisateur en cours est bien un propriétaire du fichier. Ceci est dû à la directive de configuration safe_mode.
En fonction de la configuration du serveur, en fonction du mode de fonctionnement de PHP, les droits alloués automatiquement sur les fichiers peuvent être différents.

Détection d'erreur en PHP

On défini comme étant une erreur, une anomalie au niveau de l'application. On peut donc dire qu'il y a une erreur si le programme ne fonctionne pas comme il a été conçu.
Pendant le développement, une erreur peut provenir par exemple du non respect de la syntaxe du langage, d'une mauvaise configuration, etc.
Elle peut aussi être une erreur au niveau de l'algorithme lui-même. Ainsi, notre code peut générer par exemple des prix de valeur négative ou quelque chose de ce genre. Ce qui nous pousse à faire très attention pendant la phase de conception.
De la détection des erreurs (afin de nous informer de leurs existences), et de l'information à la correction de ces erreurs, **Php nous offre la possibilité de mieux gérer ces erreurs**.

Interprétation d'un message d'erreur

Lorsque PHP détecte des erreurs, des messages explicatifs seront affichés pour les intercepter. Il nous donne notamment des informations sur le niveau d'importance de l'erreur, sur l'emplacement où elle a été détectée (nom du script et numéro de ligne).
Exemple 1
Code dans fichier.php

```
<?php
include("test.php");
echo 'Le script continue';
?>
```

Affichage à l'écran :
Warning: include(test.php) [function.include]: failed to open stream: No such file or directory in C:\wamp\www\TESTTUTO\erreur.php on line 2
Warning: include() [function.include]: Failed opening 'test.php' for inclusion (include_path='.;C:\php5\pear') in C:\wamp\www\TESTTUTO\erreur.php on line 2
Le script continue :
Exemple 2

```
<?php
require("test.php");
echo 'Le script continue';
?>
```

Affichage à l'écran
Warning: require(test.php) [function.require]: failed to open stream: No such file or directory in C:\wamp\www\TESTTUTO\erreur.php on line 2
Fatal error: require() [function.require]: Failed opening required 'test.php' (include_path='.;C:\php5\pear') in C:\wamp\www\TESTTUTO\erreur.php on line 2
Ces messages d'erreurs proviennent de l'absence du fichier test.php. En effet, l'inclusion d'un fichier inexistant sur la ligne numéro 2 génère une erreur (No such file or directory in C:\wamp\www\TESTTUTO\erreur.php on line 2).:

Les deux fonctions require et include permettent d'inclure un fichier dans un script. Le niveau d'affichage des erreurs différencie ces deux fonctions.
Dans l'exemple 1, avec un include, il n'y a que des warning à l'affichage, et le script continu à s'executer. C'est pourquoi, le texte s'affiche quand même en dessous : "Le script continue".:
Par contre dans l'exemple 2, avec la fonction require, on a un fatal error et l'exécution a été interrompue. **Il est donc important d'accorder un peu plus d'attention à ces différents niveaux d'erreur que l'on abordera par la suite**. A noter que le message d'erreurs est en général assez explicite sur le fichier et sur la ligne qui a généré l'erreur.

Les différents niveaux d'erreur

Le type d'une erreur est représenté par une constante PHP.

Message informatif (constante : E_NOTICE)

Ce type d'erreur ne s'affiche qu'à titre informatif, donc elle n'a pas beaucoup d'importance. On rencontre par exemple une erreur de ce type quand le script essaie d'accéder à une valeur qui n'a pas été déclarée, etc. L'erreur ne perturbe pas spécialement la suite du script, mais informe le développeur du problème survenu.

Message d'Alert (Constante : E_WARNING)

Comme l'on a vu dans les exemples ci-dessus, les avertissements ou warning n'interrompent pas l'exécution du script. Elles indiquent simplement un problème qui a été intercepté par le script avant (exemple : appel de la fonction ereg() avec une expression invalide)

Erreurs critique (Constante : E_ERROR)

En cas d'erreur critique de type E_ERROR, l'exécution du script est interrompu. Il s'agit d'une erreur qui ne peut pas être ignorée et intervient par exemple quand une fonctionnalité importante ne peut pas s'exécuter.

Erreurs de syntaxe (Constante E_PARSE)

Lors d'un problème d'interprétation du script, PHP produit une erreur de type E_PARSE. C'est une erreur de syntaxe dans les fichiers que l'on désigne par parse error. Cette erreur empêche le parser php de lire le code, et de l'interpréter. **Il peut s'agir par exemple de l'oubli d'un point virgule à la fin d'une instruction**.

Erreurs internes de PHP (Constantes : E_CORE_ERROR et E_CORE_WARNING)

Les erreurs E_CORE_ERROR et E_CORE_WARNING sont similaires aux erreurs E_ERROR et E_WARNING, sauf qu'elles sont générées par le code de PHP. Autrement dit, ce sont des erreurs qui ne proviennent pas forcément de PHP mais de PHP lui-même. **Les fonctions ne doivent pas générer ce genre d'erreur**.

Erreurs de compilation (Constantes :E_COMPILE_ERROR et E_COMPILE_WARNING)

Ce sont également des erreurs internes de PHP. Elles sont générées au moment de la compilation en byte code de l'application. Ces erreurs sont similaires à E_ERROR et E_WARNING, sauf qu'elles sont générées par Zend Scripting Engine.

Erreurs Utilisateur (E_USER_ERROR, E_WARNING, E_NOTICE)

Ces erreur sont comparable à E_ERROR, E_WARNING, E_NOTICE mais elles provienne de notre script. **L'utilisation de la fonction trigger_error() permet de la traiter**.

E_ALL

L'utilisation de ce niveau de rapport d'erreur provoque l'affichage de toutes des erreurs

Configuration de l'affichage des erreurs

Afficher ou ne pas afficher les erreurs

Pendant la période de développement, il est fortement utile d'afficher les messages d'erreurs pour pouvoir les détecter et les corriger. Par contre pour un site en production, si on affiche les messages d'erreur, on risque de révéler une bonne partie du secret de notre script. Et les utilisateurs de l'application se trouveront bien embêtés.
Pour éviter ce genre de problème, une petite manipulation dans le fichier php.ini permet de configurer l'affichage des messages d'erreurs. **On met la directive display_errors pour dire qu'on affiche les erreurs et on met la directive à off pour désactiver l'affichage**... sachant que sa valeur par défaut est on.
Cependant, il nous faut toujours un moyen de connaitre les éventuels problèmes de notre script. Même si on n'affiche pas les erreurs, il est possible de les connaitre en utilisant un fichier log.
On pourra donc avoir une configuration de ce type dans php.ini : display _errors

```
display _errors = on
log_errors = on
error_log = /repertoire/fichierlog.txt
```

Filtrer les erreurs : error_reporting

De la même façon on peut aussi filtrer les erreurs en fonction de leur niveau en n'affichant que les erreurs utiles. Comme l'on avait vu plus haut, chacune des constantes de type d'erreur (comme E_ERROR, etc.) présente un niveau d'erreur.
Si l'on veut par exemple afficher toutes les erreurs, on met :

```
error_reporting= E_ALL
```

Si l'on veut afficher toutes les erreurs sauf les warning, on met :

```
error_reporting = E_ALL & ~ E_WARNING
```

On peut aussi faire la même chose mais cette fois ci dans le script, en appelant directement la fonction error_reporting() comme suit :

```
error_reporting(E_ALL) ;
```

On a appris comment détecter les erreurs, et comment les interpréter. Maintenant nous allons voir comment réagir de façon simple à une erreur. Le plus souvent, les fonctions retournent NULL ou FALSE en cas d'échec. On peut faire un test et intercepter les erreurs par différentes manières :
En utilisant des conditions

```
<?php
if( ! fonction() )
{
// instructions si la fonction échoue
}
?>
```

En utilisant l'opérateur OR

```
<?php
$resultat = fonction () or intruction_erreur()
?>
```

De ma même façon on peut aussi gérer l'affichage des erreurs en fonction de l'échec ou de la réussite de la fonction. La fonction trigger_error() crée l'erreur

```
<?php
$resultat = fonction() or trigger_error ('échec de la fonction', E_USER_ERROR)
```

En mettant des "@" en début d'appel d'une fonction, on pourra aussi masquer les erreurs retournées par la fonction correspondante.
C'est une notion pas très pratique mais bonne à savoir puisque les erreurs doivent être corrigées et non pas masquées. Cependant, dans certains cas, il peut être préférable de s'assurer que le script continue à tourner, plutot que de s'arreter simplement.
Par la suite, nous allons voir de plus près comment intercepter les erreurs internes de l'application et comment réagir face à cela.

Les assertions

Une assertion c'est une sorte de règle qu'on se fixe au départ et sur laquelle est basée l'application. Il s'agit d'une infirmation fonctionnelle et logique telle que : "la note des élèves doit être un nombre positif ou null", "l'effectif des élève doit être un nombre sans décimal positif", etc.**Quand on exécute le script, les assertions seront évaluées et on a une erreur si l'une d'elles n'est pas respectée**.
L'intérêt des assertions réside sur le fait qu'elles permettent de s'assurer de la validité des données. Les assertions sont donc assimilables à des structures de contrôle. Cependant, elle est mieux adaptée à la vérification des données internes à un script. Pour les données externes, mieux vaut utiliser if.

Activer une assertion

Avant de définir une assertion, il faut d'abord s'assurer qu'elle soit active. Pour configurer l'activation de l'assertion, il existe deux façons :
La première serait de l'activer dans le code lui-même en appelant la fonction assert_options comme suit :

```
<?php
//activation de l'assertion
assert_options(ASSERT_ACTIVE, 1);
?>
```

On peut aussi configurer directement dans le fichier php.ini , sachant que :
Pour que les assertions soient prises en compte, on met :

```
<?php
assert.active = On ;
?>
```

A l'inverse, pour ignorer les assertions, on met :

```
<?php
assert.active = Off ;
?>
```

Utiliser une assertion

Après avoir activé l'assertion on peut l'utiliser. Pour cela, il faut la déclarer à l'aide de la fonction assert. Voyons ça de plus près avec des exemples
Exemple 1

```
<?php
assert_options(ASSERT_ACTIVE, 1);
$note = 4;
assert('$note>=0');
echo $note;
?>
```

Cet exemple affiche bien 4 sans problèmes à l'écran puisque la note vaut 4 donc la valeur de la note vérifie bien l'assertion.
Par contre dans l'exemple suivant, en mettant comme valeur de la note : –4 à la place de 4, le script génère un warning puisque l'assertion n'est pas vraie :
Exemple 2

```
<?php
assert_options(ASSERT_ACTIVE, 1);
$note = -4;
assert('$note>=0');
echo $note;
?>
```

Si vous tester l'exemple 2, il renvoie une erreur standard PHP de ce type :
Warning: assert() [function.assert]: Assertion "$note>0" failed in C:\wamp\www\TESTTUTO\assertion.php on line 6
Ce qui n'est pas très agréable aux yeux de l'utilisateur. Il est alors possible de personnaliser la gestion des assertions via un gestionnaire spécifique que l'on crée soit même. **Il s'agit d'une fonction possédant 3 arguments : l'adresse du script, le numéro de ligne où l'affirmation a été faite et le message.**

```
<?php
function gestion_assert($script, $ligne,$message)
{
```

```
echo "<p>Une assertion est fausse :
<strong>dans le script</strong> : $script
<strong>à la Ligne</strong> : $ligne
<strong>Message</strong> : $message</p>";
}
?>
```

On insère gestion_assert, c'est-à-dire le nom de la fonction comme étant un deuxième argument dans la fonction assert_option pour l'appeler en cas d'erreur. Sachant que ASSER_CALLBACK est la directive qui enregistre le gestionnaire d'assertion dans PHP.

```
<?php
assert_options(ASSERT_CALLBACK, 'gestion_assert');
?>
```